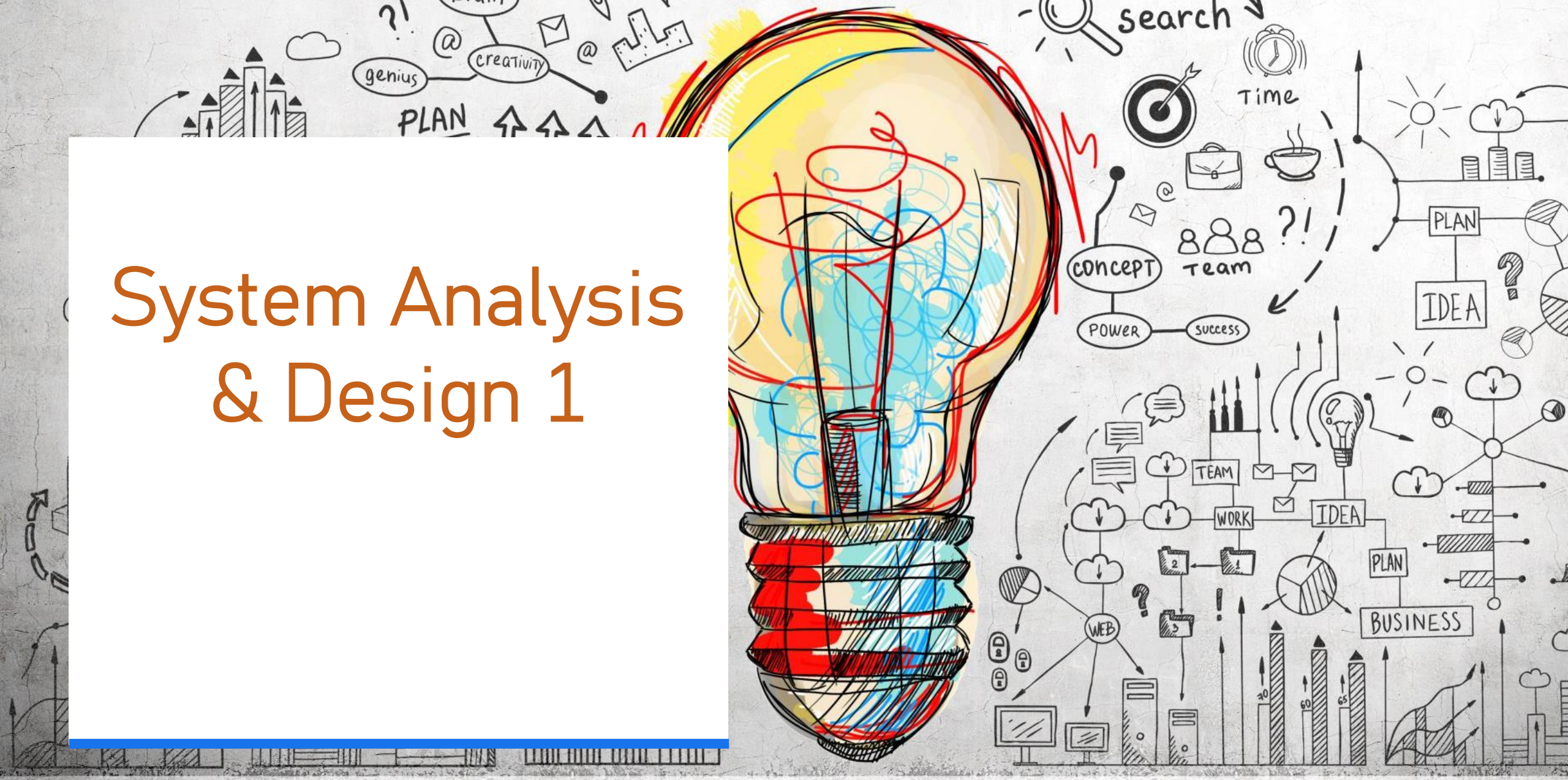
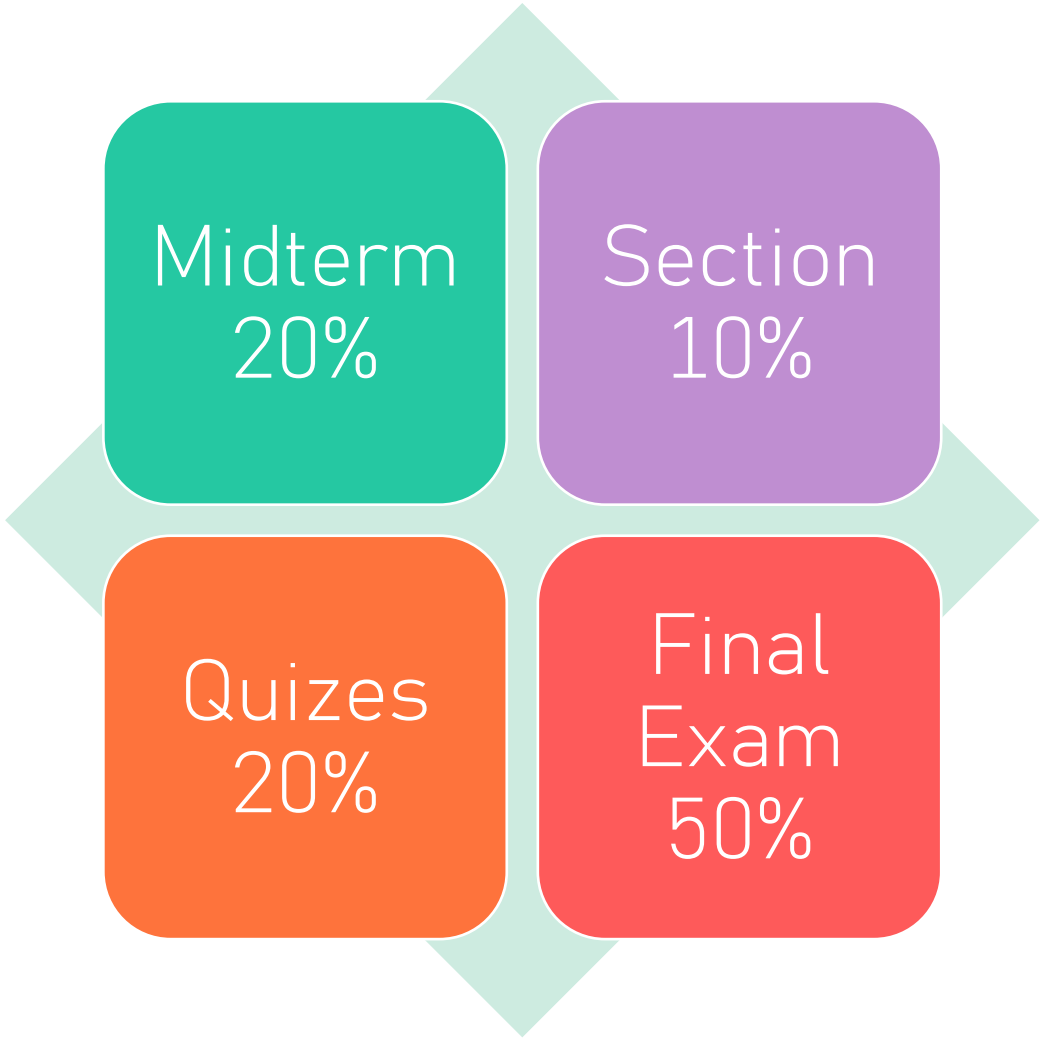
System Analysis & Design 1



Course Overview

Ch [1]	Introduction to System Analysis and Design
Ch [2]	The Planning Phase of the System Development Life Cycle (SDLC)
Ch [3]	The Analysis Phase of the System Development Life Cycle (SDLC)
Ch [4]	The Design Phase of the System Development Life Cycle (SDLC)
Ch [5]	The Implementation Phase of the System Development Life Cycle (SDLC)
Ch [6]	The Deployment and Maintenance Phase of the System Development Life Cycle (SDLC)

Course Assessment Crieteria



Chapter 1 Agenda-Part 1



What is System Analysis & Design



System Analysis & Design scope



What is system?



System Features



Concept of the System



System Types

What is System Analysis & Design (SAD)?



System Analysis and Design (SAD) is a structured process used to identify, design, implement, and maintain information systems that meet the needs of an organization.



It combines technical knowledge, analytical thinking, and business understanding to create systems that solve real-world problems.



SAD follows a systematic approach, ensuring that solutions are aligned with organizational objectives and user requirements.



What is System Analysis & Design (SAD)?

- The importance of SAD lies in its ability to reduce risks and improve efficiency in system development.

- Studies have shown that most IT project failures can be traced back to poor requirement gathering and design. By using SAD, organizations create a blueprint for building systems that are not only technically sound but also business oriented.

What is the Scope of SAD?

- The scope of SAD is vast. It spans across industries such as healthcare, banking, education, e-commerce, and government.
- For example, in healthcare, SAD is used to design patient management systems, while in banking, it supports secure online transaction systems.
- The scope also includes small-scale systems, like inventory applications, as well as enterprise-wide solutions such as ERP systems.

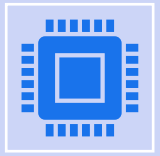
What is the Scope of SAD?



Another key aspect of the scope is adaptability.



Systems must be designed to accommodate growth and change. Today's organizations operate in dynamic environments where customer needs, regulations, and technologies evolve rapidly.



SAD provides the foundation to design systems that are flexible, scalable, and capable of adapting to future changes.

What is the Scope of SAD?

1

SAD is not just about building software; it is about aligning technology with organizational goals.

2

Its importance grows as businesses increasingly depend on digital solutions to remain competitive.

3

A well-designed system can be the difference between organizational success and failure.

Simply what is a system?



A system can be defined as a set of interrelated components that work together according to sets of defined steps to achieve a common goal.



These components could be physical elements, like machines and hardware, or abstract ones, such as processes, rules, and procedures.

System Features

01 Organization

02 Interaction

03 Interdependence

04 Integration

05 Central Objective

System Features

Organization means that A system is structured, with components arranged in a way that facilitates cooperation and coordination.

This organization ensures that all parts contribute toward the overall objectives.

For example, in a university system, departments, faculties, and administration are structured to collectively achieve educational goals.

System Features

Interaction means that the components of a system are not isolated but interact to process inputs into outputs.

These interactions may be physical (machines working together) or logical (information sharing between subsystems).

Without interaction, the system would be just a collection of disconnected parts rather than a functional whole.

System Features

Interdependence indicates that each subsystem or component relies on others to perform its function.

For example, in an airline reservation system, the ticketing subsystem depends on the seat allocation subsystem, which in turn depends on the database subsystem.

Failure in one affects the performance of others.

System Features

The fourth characteristic is **Integration** which refers to the way the different components of a system work together as a unified whole.

A system is not just a collection of independent parts; instead, it is designed so that its subsystems complement one another.

For example, in an enterprise information system, the finance, sales, and inventory subsystems must integrate seamlessly to provide accurate real-time data.

If these components operate in isolation without integration, the system's overall effectiveness is compromised.

System Features



This characteristic highlight that changes in one part of a system often affect others. In an integrated healthcare system, for instance, when a doctor updates a patient's medical record, the changes should automatically reflect in the billing, pharmacy, and insurance subsystems.

This interconnection prevents duplication of work and ensures data consistency across the entire system.

Integration also promotes efficiency and coordination. By designing subsystems to work together, organizations reduce redundancy, minimize errors, and streamline operations.

For example, modern ERP (Enterprise Resource Planning) systems integrate finance, HR, and supply chain into a single platform, eliminating the need for separate disconnected systems.

System Features

Every system is established with a **central objective**, which acts as the guiding purpose behind its design and operation. Without a central objective, a system would lack direction and meaning, making it impossible to evaluate whether the system is functioning successfully.

For example, the central objective of an online banking system is to provide secure and convenient financial transactions for customers.

The central objective serves as the system's **benchmark for performance**. When evaluating a system, analysts ask: "Is the system achieving its intended purpose?" If the system meets or exceeds its central objective, it is considered effective.

For instance, if a supply chain management system reduces delivery delays as intended, then it has achieved its objective.

System Features

This characteristic also provides **alignment across subsystems**. Each subsystem within a larger system must contribute toward achieving the central objective.

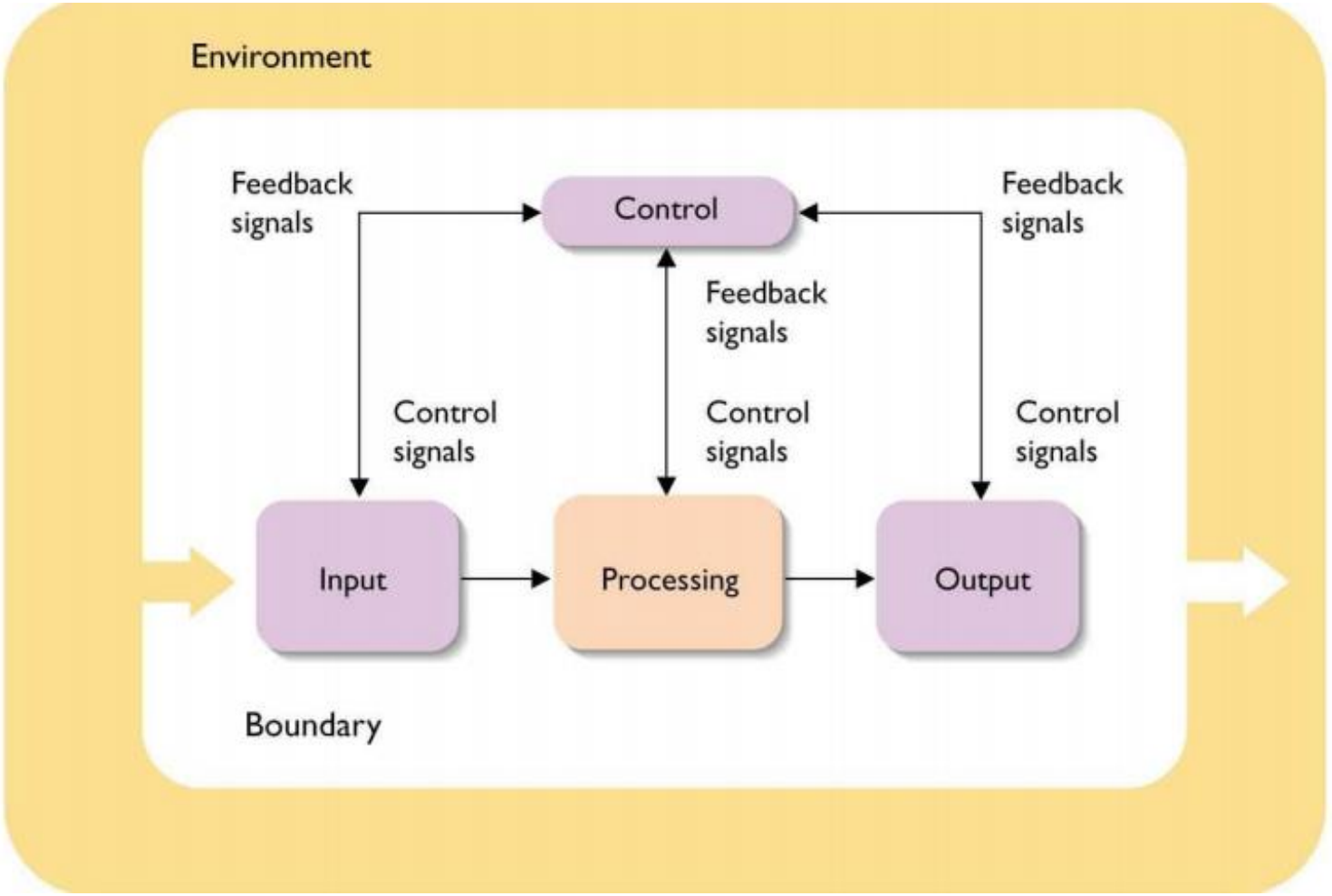
For example, in a university system, the academic, administrative, and library subsystems all serve the overarching goal of providing quality education and research support. If subsystems do not align with the central objective, they may create inefficiencies or conflicts.

Concept of the System

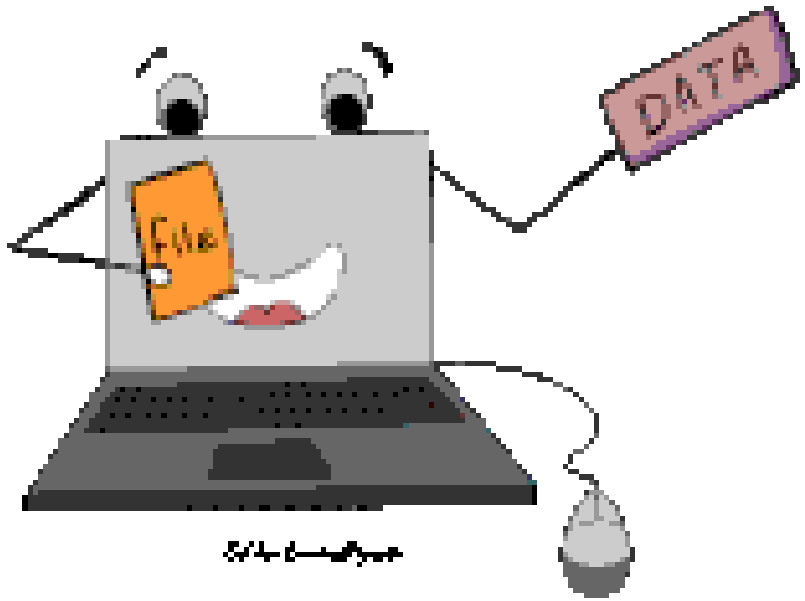
- Beyond the fundamental traits such as organization, interaction, interdependence, integration, and a central objective, systems also exhibit additional features (Known as Basic concepts for any system) that define how they operate and interact with their surroundings.

- This Concept describes the flow of resources and information within the system, its relationship with external factors, and its ability to self-regulate.

Concept of the System

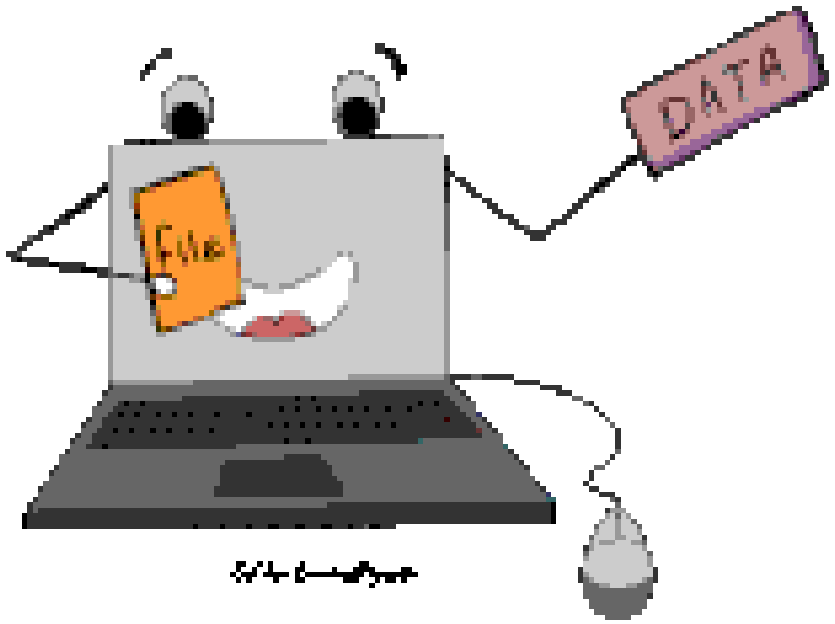


Concept of the System



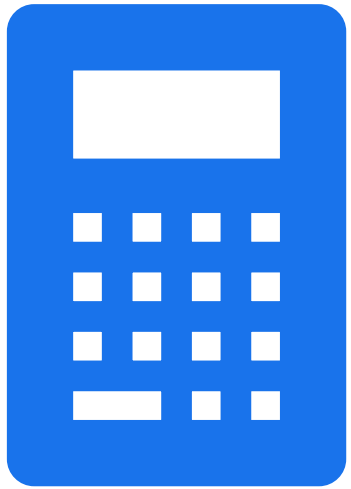
- Inputs are the resources, data, or energy that a system requires to function. They represent the raw materials fed into the system for transformation.
- In an information system, inputs often take the form of data entered by users, such as student grades in an academic system or customer details in an e-commerce application.
- Without inputs, a system cannot begin its operation because it lacks the foundation needed to generate outputs.

Concept of the System



- Inputs can be categorized into different types, such as manual inputs provided directly by users or automated inputs gathered from sensors, machines, or other systems.
- For example, a weather monitoring system receives automated data inputs from sensors measuring temperature, humidity, and wind speed.
- The quality and accuracy of inputs directly influence the reliability of the outputs produced by the system.
- Another key feature of inputs is that they often require validation before processing. Systems incorporate validation checks to ensure that only correct, complete, and authorized data enters the system. This minimizes errors and prevents inaccurate results.
- For instance, an online payment system verifies card numbers and expiration dates before processing transactions.

Concept of the System



- Processing refers to the set of operations that transform inputs into meaningful outputs.
- For example, in a payroll system, employee attendance data (input) is processed through rules for overtime, deductions, and bonuses to generate net salary information (output).
- Processing can be simple or complex depending on the system. Simple processing might involve basic arithmetic, such as totaling monthly sales figures. Complex processing, on the other hand, could include running predictive analytics using machine learning algorithms to forecast future customer demand.
- The complexity of processing often determines the system's performance and value to the organization. Effective processing requires both speed and accuracy. Systems must be designed to handle processing workloads efficiently without delays, while also ensuring that business rules and logic are consistently applied.
- Inaccurate processing, such as misapplied formulas in financial systems, can lead to serious organizational consequences.

Concept of the System



Outputs are the final products, results, or services that a system delivers after processing inputs.



In an information system, outputs are often reports, dashboards, or documents that provide decision-makers with actionable insights.



For example, in a hospital management system, outputs include patient records, billing invoices, and appointment schedules.



Outputs must be meaningful and relevant to the users they serve. If a system produces outputs that are difficult to understand or irrelevant, its usefulness decreases. Therefore, system designers focus not only on accuracy but also on the usability and presentation of outputs.

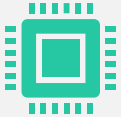
Concept of the System



Outputs are the final products, results, or services that a system delivers after processing inputs.



To keep output meaningful, data visualization tools transform raw numbers into charts and graphs to make outputs more comprehensible.



Outputs can also take non-physical forms, such as automated alerts, notifications, or even machine actions. For example, a traffic control system may output a red-light signal to regulate vehicle flow.



Regardless of form, the quality of outputs reflects the effectiveness of the entire system and determines user satisfaction.

Concept of the System



Every system operates within defined boundaries that separate it from its external environment.



Boundaries determine what is part of the system and what lies outside. For instance, in a university information system, course registration, grading, and library services may be within the system's boundaries, while government education policies belong to the external environment.



Boundaries help analysts focus on the relevant parts of a system during design and development.

Concept of the System



The environment of a system includes all external factors that influence or interact with it. This could involve customers, suppliers, competitors, regulations, or even other systems.



For example, an e-commerce system interacts with external payment gateways, logistics providers, and regulatory authorities.



These environmental factors shape how the system functions and adapt over time.



Boundaries and environments are not always rigid; they can evolve as the system changes. For instance, a banking system initially focused only on branch-level transactions may expand its boundaries to include online and mobile banking due to changes in the external environment.



Recognizing this dynamic relationship helps organizations design systems that are flexible and adaptable to new challenges.

Concept of the System

Feedback is the information a system receives about its performance (Output), while control is the mechanism that uses this feedback to guide system behavior.

Together, they ensure that the system stays aligned with its objectives. For example, in a thermostat-controlled heating system, temperature sensors provide feedback, and the thermostat adjusts heating levels to maintain the desired temperature.

Feedback can be positive or negative. Positive feedback reinforces and strengthens system behavior, while negative feedback signals problems that require corrective action.

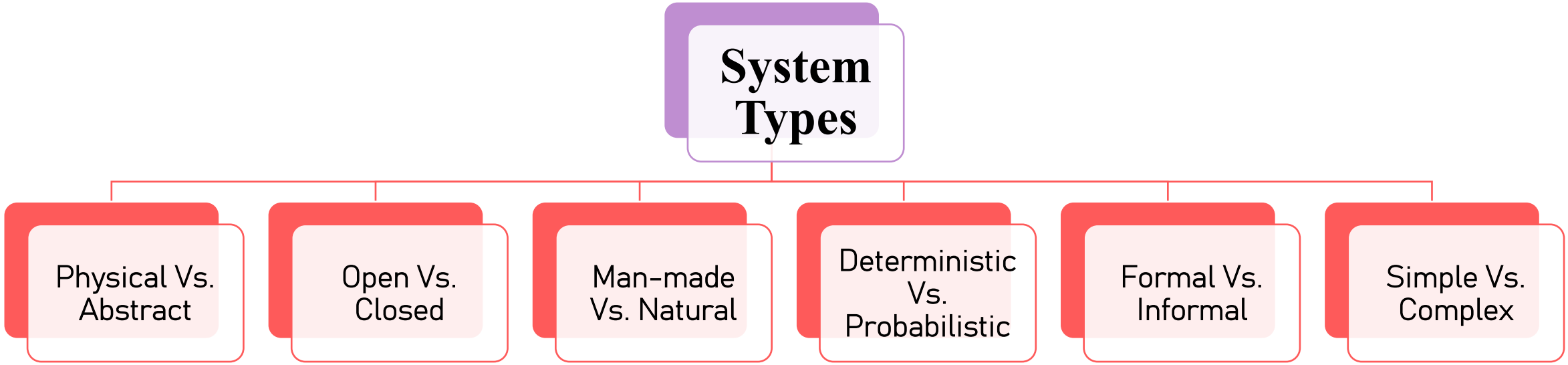
Concept of the System

By analyzing both, the system can adjust processes to improve performance. Control mechanisms establish rules and standards for how the system should respond to feedback.

In organizations, this could mean setting thresholds for performance indicators, such as maximum error rates or minimum response times.

If actual outputs deviate from these standards, corrective actions are triggered. For example, in manufacturing, feedback from quality checks ensures defective products are identified and adjustments are made to maintain standards.

System Types



System Types

One broad category is **physical vs. abstract systems**.

Physical systems are tangible and can be touched, such as computer hardware or a manufacturing assembly line.

Abstract systems are conceptual, such as mathematical models or organizational structures, which exist in theory but guide real-world operations.

System Types

Another classification is **open vs. closed systems**.

Open systems interact with their environment, exchanging inputs and outputs, while closed systems operate in isolation.

Most real-world systems, especially information systems, are open because they rely on external data and provide outputs to users or other systems.

For example, an e-commerce system interacts with users, banks, and logistics partners, making it an open system.

System Types

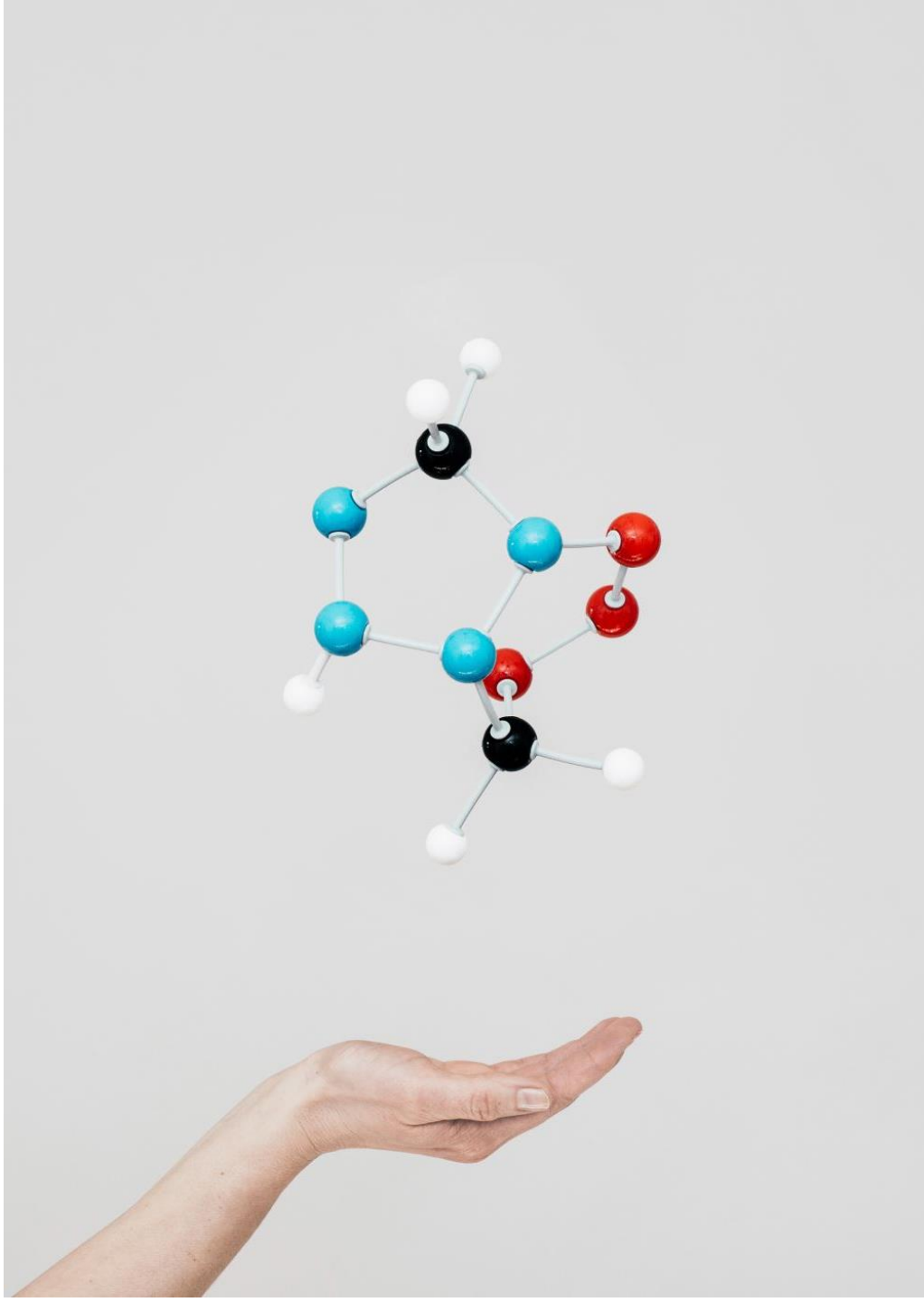
Systems can also be categorized as **man-made vs. natural systems**.

Natural systems, like ecosystems or the human body, occur without human intervention.

Man-made systems, such as traffic control systems or ERP software, are designed and controlled by humans to achieve specific goals. In system design, we focus primarily on man-made systems.

System Types

- Another type is deterministic vs. probabilistic systems.
- Deterministic systems behave predictably and produce the same output given the same input, such as a calculator.
- Probabilistic systems, on the other hand, include elements of uncertainty or randomness.
- For example, stock market forecasting systems are probabilistic because they rely on uncertain data and unpredictable variables.



System Types



System can be Formal vs. Informal systems.



Formal systems are structured and well-documented systems that operate according to clearly defined rules, procedures, and standards. They are designed to ensure consistency, fairness, and reliability across different scenarios.



Examples include payroll systems, academic grading systems, or banking transaction systems, where accuracy, compliance, and uniformity are essential.



Because they are rule-based and documented, formal systems can be audited, evaluated, and replicated easily, making them suitable for industries where regulation and accountability are critical.

System Types



However, Informal systems, on the other hand, are not based on official documentation or strict rules.



They rely on personal interactions, relationships, trust, and shared understandings that emerge naturally within organizations or communities.



For example, when employees collaborate through casual discussions, social networks, or mentorship outside formal procedures, they are using an informal system.



These systems are highly flexible, adaptive, and often faster at solving unexpected problems compared to formal ones, making them crucial for innovation and quick decision-making.

System Types

Finally, systems can be classified as **simple vs. complex systems**.

Simple systems have few components and straightforward interactions, while complex systems involve multiple interdependent subsystems with high levels of interaction.

For example, a library book tracking system is relatively simple compared to a national defense system, which integrates multiple technologies and organizations. Understanding these classifications helps analysts choose the right tools and methods for system design.

System Types

- Information Systems (IS) are considered a **type of system**.
- In fact, most of *System Analysis and Design* is centered around them.
- An information system can be seen as a man-made system designed to collect, process, store, and distribute information to support decision-making, coordination, control, analysis, and visualization in an organization.



System Types

- Information Systems (IS) belong to multiple system classifications simultaneously
- They are man-made systems because they are intentionally designed by humans. They are typically open systems, since they interact with external entities like customers, suppliers, or government regulations.
- Depending on their scope, they can be simple (like a library catalog system) or complex (like global airline reservation systems). Some are deterministic, producing predictable outputs like payroll calculations, while others are probabilistic, such as forecasting demand in retail.
- This flexibility shows that information systems are not confined to a single type but rather reflect a hybrid of system characteristics depending on their purpose and design.

System Types



Information Systems (IS) don't exist in isolation — information systems can embody any combination of the other system types (formal/informal, open/closed, simple/complex, etc.), depending on their design and purpose.

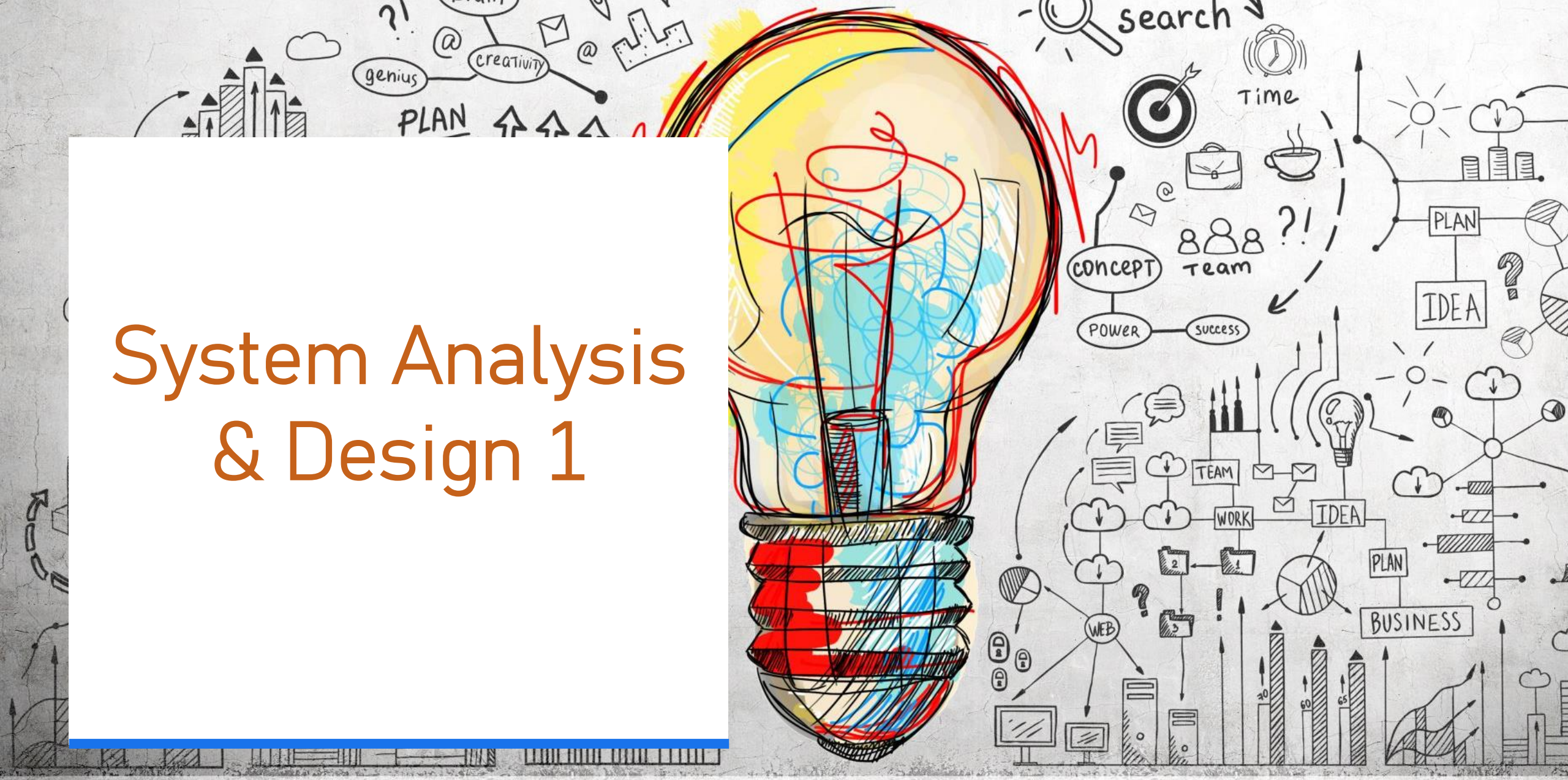


For example, a payroll information system is formal, man-made, deterministic, and relatively simple, while a global supply chain management system is man-made, open, complex, and may include probabilistic forecasting.

See You Aga



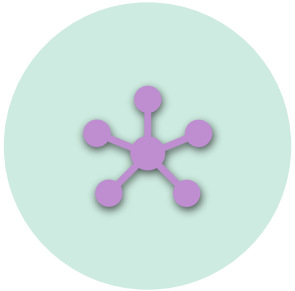
System Analysis & Design 1



Chapter 1 Agenda-Part 2



Information System



Dimensions of Information System



Overview of System Development lifecycle



Introduction of Development Methodologies

System Types

- Information Systems (IS) are considered a **type of system**.
- In fact, most of *System Analysis and Design* is centered around them.
- Information Systems (IS) belong to **multiple system classifications simultaneously**



System Types

- ✓ An ***information system*** is a collection of *hardware, software, data, people and procedures* that are designed to generate information that supports the *day-to-day*, *short-range*, and *long-range activities* of users in an organization.



- ✓ Information systems can be classified into five categories: *office information systems, transaction processing systems, management information systems, decision support systems, and expert systems*.

Transaction Processing Systems

A transaction processing system *(TPS)* is an information system that *captures and processes data* generated during an organization's *day-to-day transactions*.

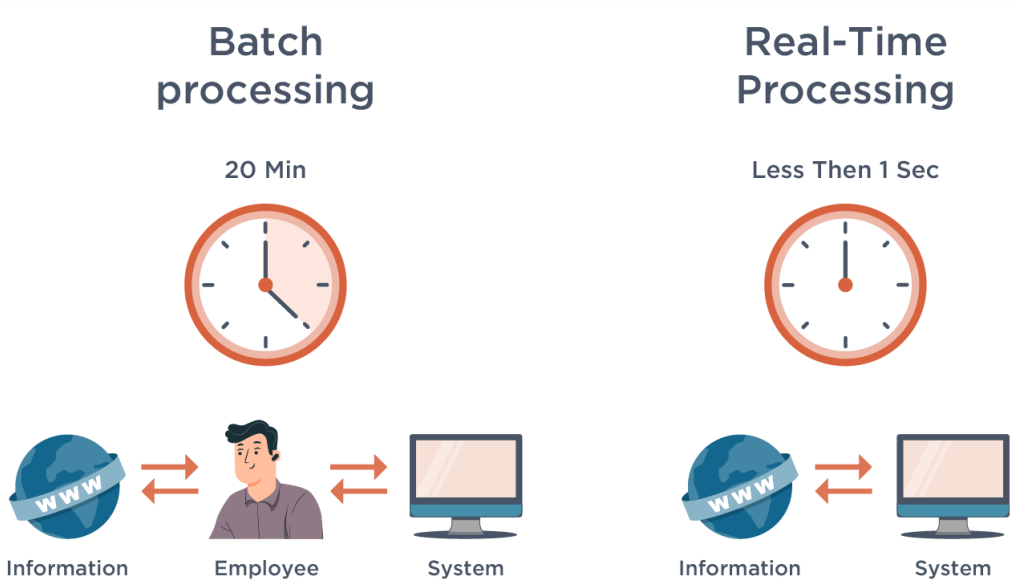
A transaction is a business activity such as a deposit, payment, order or reservation.

1. Recording a business activity such as a **student's registration**, a **customer's order**, or a **client's payment**.

2. Confirming an action or triggering a response, such as **printing a student's schedule**, sending a **thank-you note to a customer**, generating an employee's **paycheck** or **issuing a receipt to a client.**

3. Maintaining data, which involves **adding new data**, **changing existing data**, or **removing unwanted data**.

Transaction Processing systems



- *These transactions can be categorized into :*
- batch transaction processing, and real time transaction processing.

Transaction Processing Systems

01

The first transaction processing systems usually used batch processing. With batch processing, *transaction data is collected over a period, and all transactions are processed later*, as a group.

02

With online transaction processing (OLTP) the computer *processes transactions as they are entered.*

03

Example, When you register for classes, your school probably uses OLTP. The registration administrative assistant enters your desired schedule, and the computer immediately prints your statement of classes. The invoices,

Transaction Processing Systems



Today, most transaction processing systems use online transaction processing.



Some routine processing tasks such as calculating ***paychecks or printing invoices***, however, are performed more effectively on a ***batch basis***. For these activities, many organizations still use batch processing techniques.

Management Information Systems

- A management information system, or MIS, is an information system that generates accurate, timely and organized information so managers and other users can make decisions, solve problems, supervise activities, and track progress

- Because it generates reports on a regular basis, a management information system sometimes is called a management reporting system (MRS).



Management Information Systems

- MIS often are integrated with TPS and generate information that management and other users need to perform their jobs.
- To process a sales order, for example, the transaction processing system records the sale, updates the customer's account balance, and makes a deduction from inventory. Using this information, the related management information system can produce reports that recap daily sales activities; list customers with past due account balances; graph slow or fast selling products; and highlight inventory items that need reordering.



Dimensions of Information Systems



An information system is a structured system designed to manage the flow of information by collecting inputs, processing them into meaningful outputs, and delivering them to the right users at the right time.



Its core purpose is to support decision-making, enhance efficiency, and enable communication within organizations. Examples include payroll systems, hospital management systems, e-commerce platforms, and enterprise resource planning (ERP) systems.

Dimensions of Information Systems

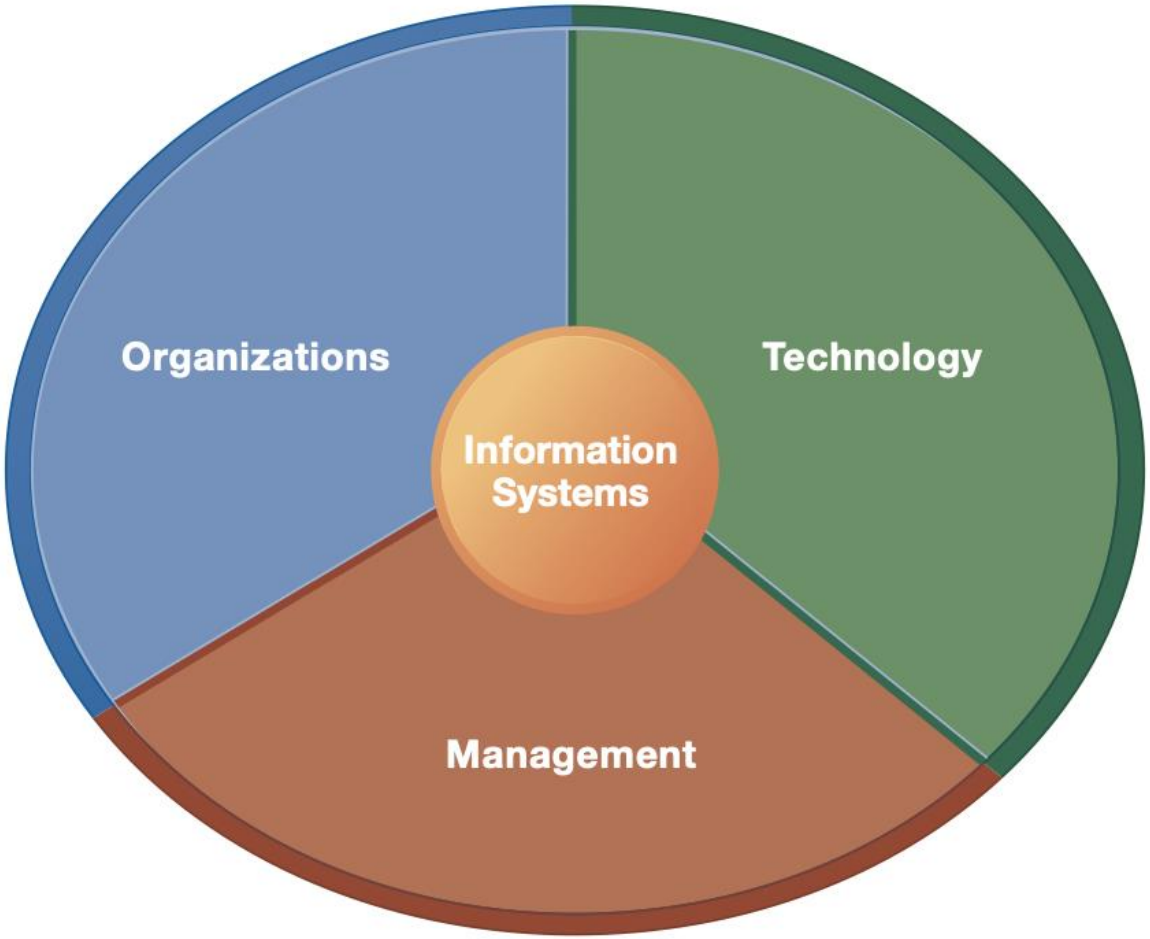


Information systems combine both technical and human components. They are not purely technological; instead, they integrate people, processes, and technology to achieve organizational goals.



The information systems combine three different dimensions like organization, Management, and Information Technology.

Dimensions of Information Systems





Dimensions Of Information Systems

1-Organizations

- The key elements of an organization are its

1-people,

2-structure,

3-business processes,

4-politics,

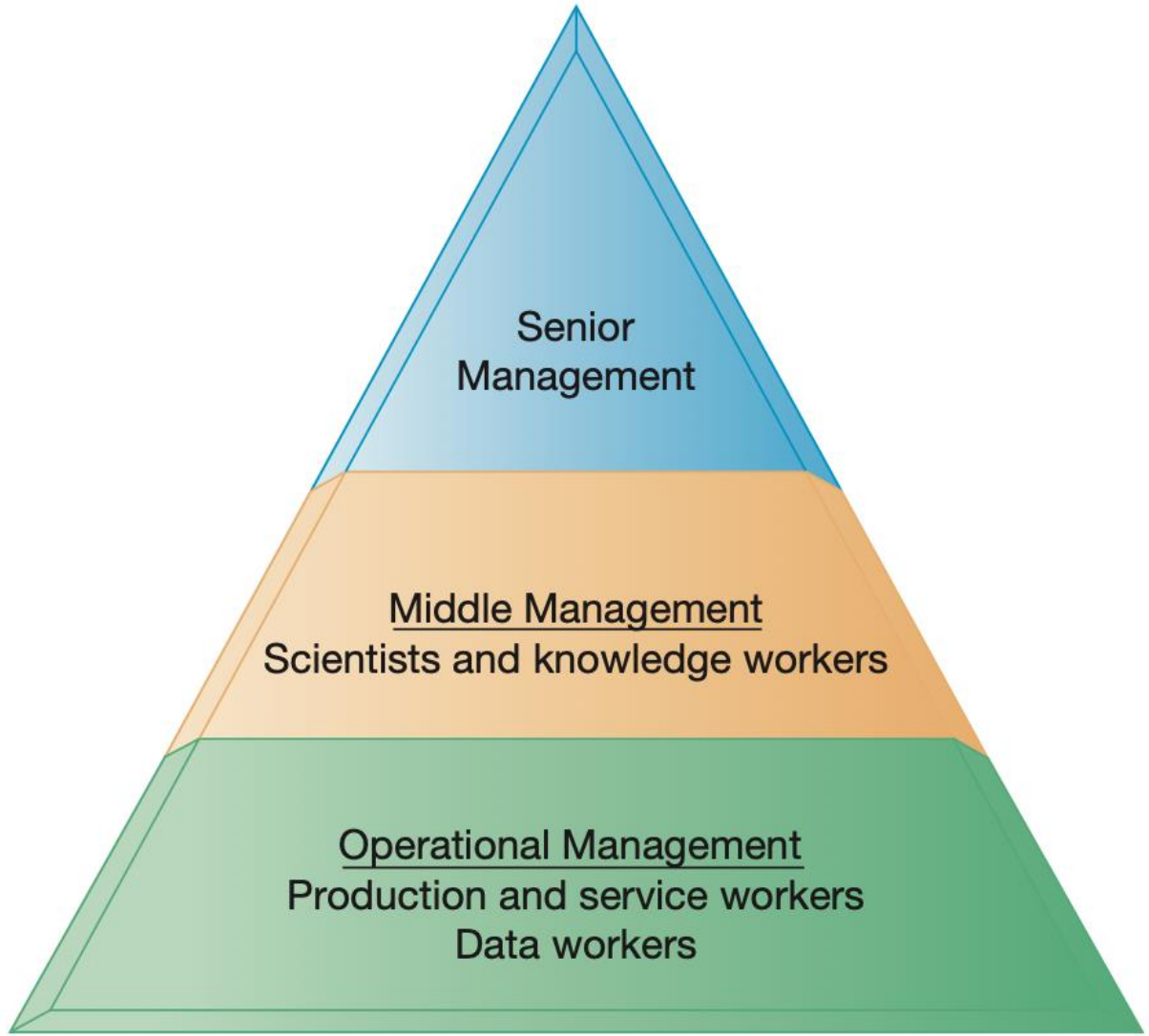
5-and culture



Dimensions Of Information Systems

1-Organizations

- Organizations have a structure that is composed of different levels and specialties.
- Authority and responsibility in a business firm are organized as a hierarchy, or a pyramid structure.
- The upper levels of the hierarchy consist of managerial, professional, and technical employees, whereas the lower levels consist of operational personnel.


Senior
Management
Middle Management
Scientists and knowledge workers
Operational Management
Production and service workers
Data workers

Dimensions of information systems

1-Organizations

1-Senior management makes long-range strategic decisions about products and services as well as ensures financial performance of the firm.

Experts are employed and trained for different business functions.

2-Middle management carries out the programs and plans of senior management, and

Dimensions of information systems

1-Organizations

Knowledge workers , such as engineers, scientists, or architects, design products or services and create new knowledge for the firm. They are employees whose main job is to think, create, and develop ideas rather than just doing routine tasks.

- For example, an engineer might design a new type of car. A scientist might discover a new medicine. An architect might design a new building.
- Their work creates new knowledge or new products that help the company grow, improve, and stay competitive. They use their brains and skills to build the future of the company.

Dimensions of information systems

1-Organizations

3-Operational management is responsible for monitoring the daily activities of the business.

Whereas data workers , such as secretaries or clerks, assist with scheduling and communications at all levels of the firm. Production or service workers produce the product and deliver the service.

❖ The information system must reflect each level responsibilities through business processes.

Dimensions of information systems

1-Organizations

- An organization coordinates work through its hierarchy and through its business processes. Most organizations' business processes include formal rules that have been developed over a long time for accomplishing tasks.
- These rules guide employees in a variety of procedures, from writing an invoice to responding to customer complaints. Some of these business processes have been written down, but others are informal work practices, such as a requirement to return telephone calls from coworkers or customers, that are not formally documented.
- Information systems **automate many business processes**. For instance, how a customer receives credit or how a customer is billed is often determined by an information system that incorporates a set of formal business processes.

Dimensions of information systems

1-Organizations

- Each organization has a unique culture , or fundamental set of assumptions, values, and ways of doing things, that has been accepted by most of its members.
- Parts of an organization's culture can always be found embedded in its information systems. For instance, Provide company-wide chat channels or "Ask-the-CEO" portals where anyone can post questions that leaders answer.

Two-pizza" rule No meeting should be so big that two pizzas can't feed everyone

Celebrate small wins

Dimensions of information systems

1-Organizations

Organizational politics are the unofficial, behind-the-scenes efforts that people make to gain power, protect their interests, or influence decisions inside a company.

- It shows up as alliance-building, lobbying a boss before a meeting, controlling information flow, or trading favors.

Dimensions of information systems

1-Organizations

Different levels and specialties in an organization create different interests and points of view. These views often conflict over how the company should be run and how resources and rewards should be distributed. Conflict is the basis for organizational politics. Information systems come out of this cauldron of differing perspectives, conflicts, compromises, and agreements that are a natural part of all organizations.

Dimensions of information systems

2-Management

- Management's job is to make sense out of the many situations faced by organizations, make decisions, and formulate action plans to solve organizational problems.
- Managers perceive business challenges in the environment, they set the organizational strategy for responding to those challenges, and they allocate the human and financial resources to coordinate the work and achieve success.

Dimensions of information systems

2-Management

- But managers must do more than manage what already exists. They must also create new products and services and even re-create the organization from time to time.
- A substantial part of management responsibility is creative work driven by new knowledge and information.

3-Information technology can play a powerful role in helping managers design and deliver new products and services and redirecting and redesigning their organizations.



What is information Technology?

- Information technology (IT) consists of all the hardware and software that a firm needs to use to achieve its business objectives.
- This includes not only computer machines, storage devices, and handheld mobile devices but also software, such as the Windows or Linux operating systems, the Microsoft Office desktop productivity suite, and the many thousands of computer programs that can be found in a typical large firm.

Dimensions of information systems

3-Information Technology

- Information technology is one of many tools managers use to cope with change. Computer hardware is the physical equipment used for input, processing, and output activities in an information system. It consists of the following: computers of various sizes and shapes (including mobile handheld devices); various input, output, and storage devices; and telecommunications devices that link computers together.
- Computer software consists of the detailed, preprogrammed instructions that control and coordinate the computer hardware components in an information system.

Dimensions of information systems

3-Information Technology

Networking and telecommunications technology , consisting of both physical devices and software, links the various pieces of hardware and transfers data from one physical location to another.

- Computers and communications equipment can be connected in networks for sharing voice, data, images, sound, and video. A network links two or more computers to share data or resources, such as a printer.
- The world's largest and most widely used network is the Internet . The Internet is a global "network of networks" that uses universal standards to connect millions of networks in more than 230 countries around the world.

Dimensions of information systems

3-Information Technology

- Internal corporate networks based on Internet technology are called intranets .
- Private intranets extended to authorized users outside the organization are called extranets , and firms use such networks to coordinate their activities with other firms for making purchases, collaborating on design, and other interorganizational work.
- For most business firms today, using Internet technology is both a business necessity and a competitive advantage.

Dimensions of information systems

3-Information Technology

- The **World Wide Web** is a service provided by the Internet that uses universally accepted standards for storing, retrieving, formatting, and displaying information in a page format on the Internet.
- Web pages contain text, graphics, animations, sound, and video and are linked to other web pages. By clicking on highlighted words or buttons on a web page, you can link to related pages to find additional information and links to other locations on the web. The web can serve as the foundation for new kinds of information systems.

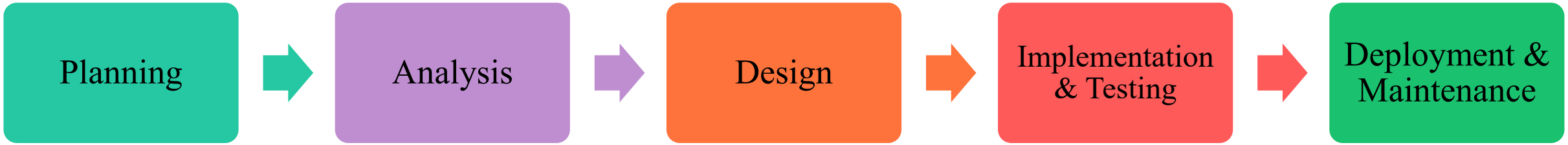
Dimensions of information systems

3-Information Technology

- All these technologies, constitute the firm's information technology (IT) infrastructure.
- The IT infrastructure provides the foundation, or platform , on which the firm can build its specific information systems. Each organization must carefully design and manage its IT infrastructure so that it has the set of technology services its needs for the work it wants to accomplish with information systems.

Overview of the System Development Life Cycle (SDLC)

- The System Development Life Cycle (SDLC) provides a structured framework for building systems, ensuring that development follows a clear path from conception to deployment.
- The SDLC typically includes five stages such as **planning, analysis, design, implementation, testing, deployment, and maintenance.** Each phase has specific deliverables and goals, creating checkpoints that minimize risks.



Overview of the System Development Life Cycle (SDLC)

1-Planning Phase

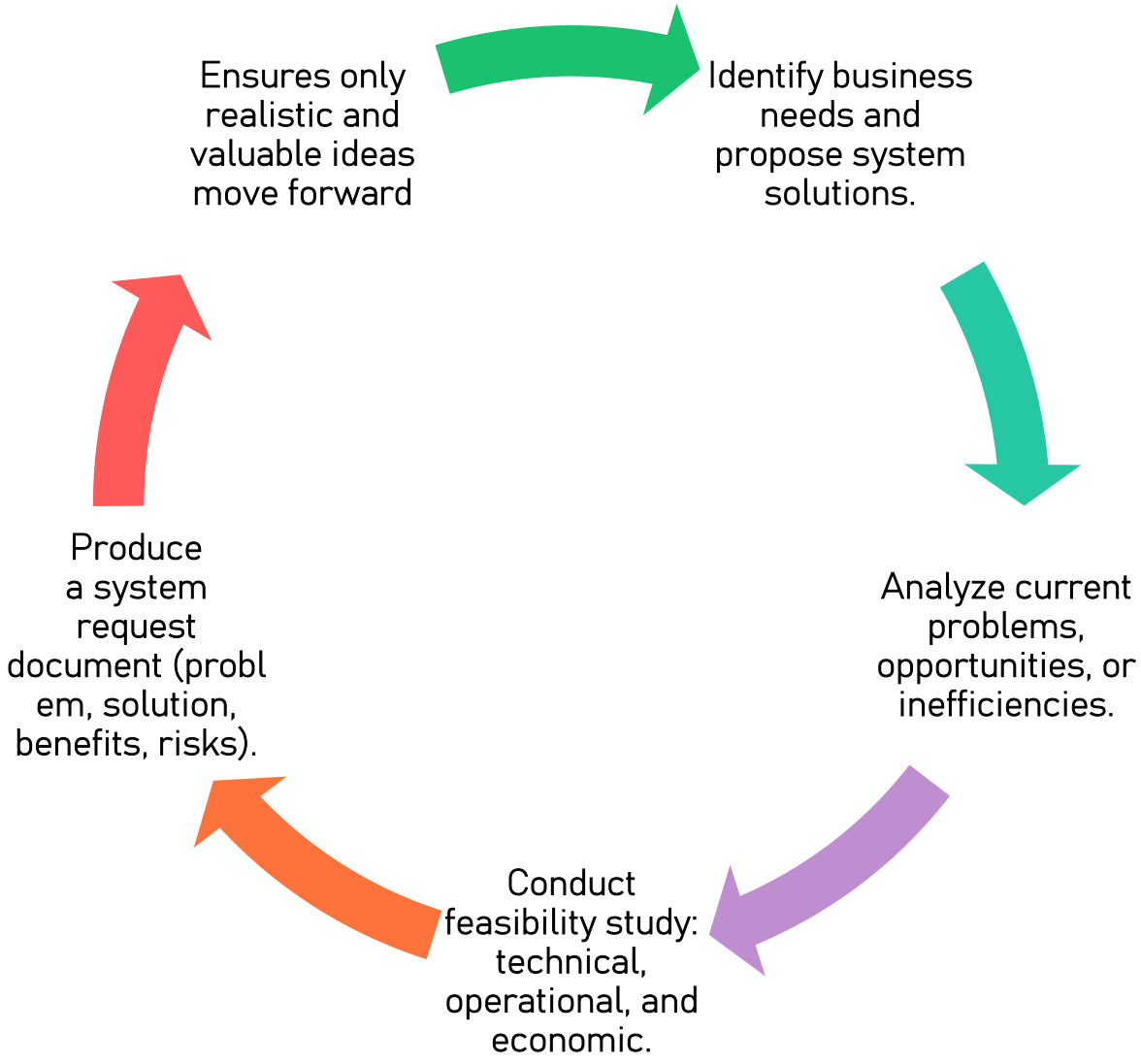
Purpose of Planning Phase

- Defines project scope, feasibility, and objectives.
- Ensures alignment with business strategies.
- Prevents wasting resources on non-viable projects



Overview of the System Development Life Cycle (SDLC)

Planning Phase
Project Initiation



Overview of the System Development Life Cycle (SDLC)

Planning Phase
Project Management



Begins after project approval.



Develop project plan (tasks, schedule, milestones, budget, resources).



Project manager coordinates team, assigns responsibilities, and monitors progress.



Includes risk management: identify risks (budget, delays, scope changes) and plan mitigations.



Maintain clear communication with stakeholders.



Keeps project on track and increases success chances

Overview of the System Development Life Cycle (SDLC)

2-Analysis Phase

Purpose of Analysis Phase

01

Understand user requirements in depth.

02

Ensure the system solves real problems, not assumptions.

Overview of the System Development Life Cycle (SDLC)

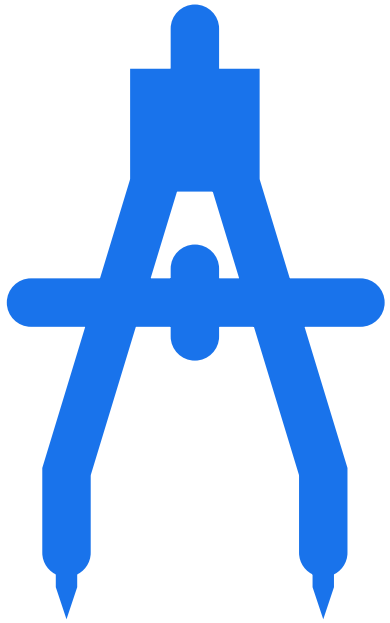
Key Activities

1-Requirements Gathering

- Collect information about user needs and expectations.
- Methods: interviews, surveys, document review, observation.
- Output: **requirements specification document** including:
  - *Functional requirements* (what the system does).
  - *Non-functional requirements* (performance, security, usability, etc.).
- Prioritize requirements.

3- Design Phase

Overview of the System Development Life Cycle (SDLC)



- Purpose of Design Phase
- Convert requirements into a detailed system blueprint.
- Bridge between analysis and actual development.

Main Components

- Logical Design
  - Focuses on *what* the system should do (not how).
  - Defines data structures, workflows, and user interfaces conceptually.
  - Includes logical database schemas and UI mock-ups.
  - Ensures all requirements are addressed before technical choices are made.
  - Lets stakeholders validate functionality without technical details.



Overview of the System Development Life Cycle (SDLC)

- Physical Design
  - Focuses on *how* the system will be implemented.
  - Converts logical models into detailed technical specifications.
  - Defines actual database schema, programming languages, hardware, and architecture (e.g., client-server, cloud).
  - Covers network setup, security, and performance strategies.
  - Provides developers with a precise blueprint for coding, database creation, and integration.
  - Reduces errors and ensures system meets functional and performance expectations.

4- Implementation & Testing Phase

Overview of the System Development Life Cycle (SDLC)

Purpose of Implemetation and Testing

- Build the system (implementation).
- Verify it works correctly and meets requirements (testing).

Implementation (System Construction)

- Programmers write code, create databases, and integrate components.
- Developers follow design specifications for accuracy.
- Tools used: programming frameworks, IDEs.
- Documentation is prepared for future maintenance and user training.
- **Unit testing**: test individual modules to ensure they function correctly.

Overview of the System Development Life Cycle (SDLC)

Testing

- Goes beyond unit testing to ensure full system quality.
- Types of testing:
  - Integration testing → verify modules work together.
  - System testing → check the system.
  - User Acceptance Testing (UAT) → end users confirm requirements are met.
- Testing is iterative; issues may require redesign or recoding.
- Ensures system is stable, secure, and ready for deployment.
- Reduces risks of failure and builds confidence before launch.

Overview of the System Development Life Cycle (SDLC)

5- Deployment & Maintenance Phase

Deployment Phase

- **Purpose**: Release system to live environment and ensure smooth operation.

Key Activities:

- **System installation**: data migration, configuring hardware/software, and rolling out the system.
- **Deployment strategies**:
  - *Parallel installation* → old and new systems run together.
  - *Direct cutover* → immediate switch to the new system.
  - *Phased rollout* → gradual introduction.
- **User training**: workshops, manuals, e-learning, help desks.
- **Documentation**: user-friendly guides for use and troubleshooting.
- **Goal**: reduce resistance, build user confidence, and maximize system value.

Overview of the System Development Life Cycle (SDLC)

Maintenance Phase

- **Purpose:** Provide ongoing support and ensure long-term system value.

Types of Maintenance:

Corrective: fix bugs/errors found after deployment.

Adaptive: modify system for new regulations, technologies, or business needs.

Perfective: add features, improve performance, based on user feedback.

Preventive: prevent future issues by optimizing code, upgrading hardware, enhancing security.

- **Goal:** keep system accurate, secure, efficient, and aligned with organizational goals.
- **Big Idea:** Maintenance is about evolving the system—not just fixing problems.



Systems Development Methodologies

SDM

Structured Design



Rapid Application Development (RAD)

Agile Development

Structured Design

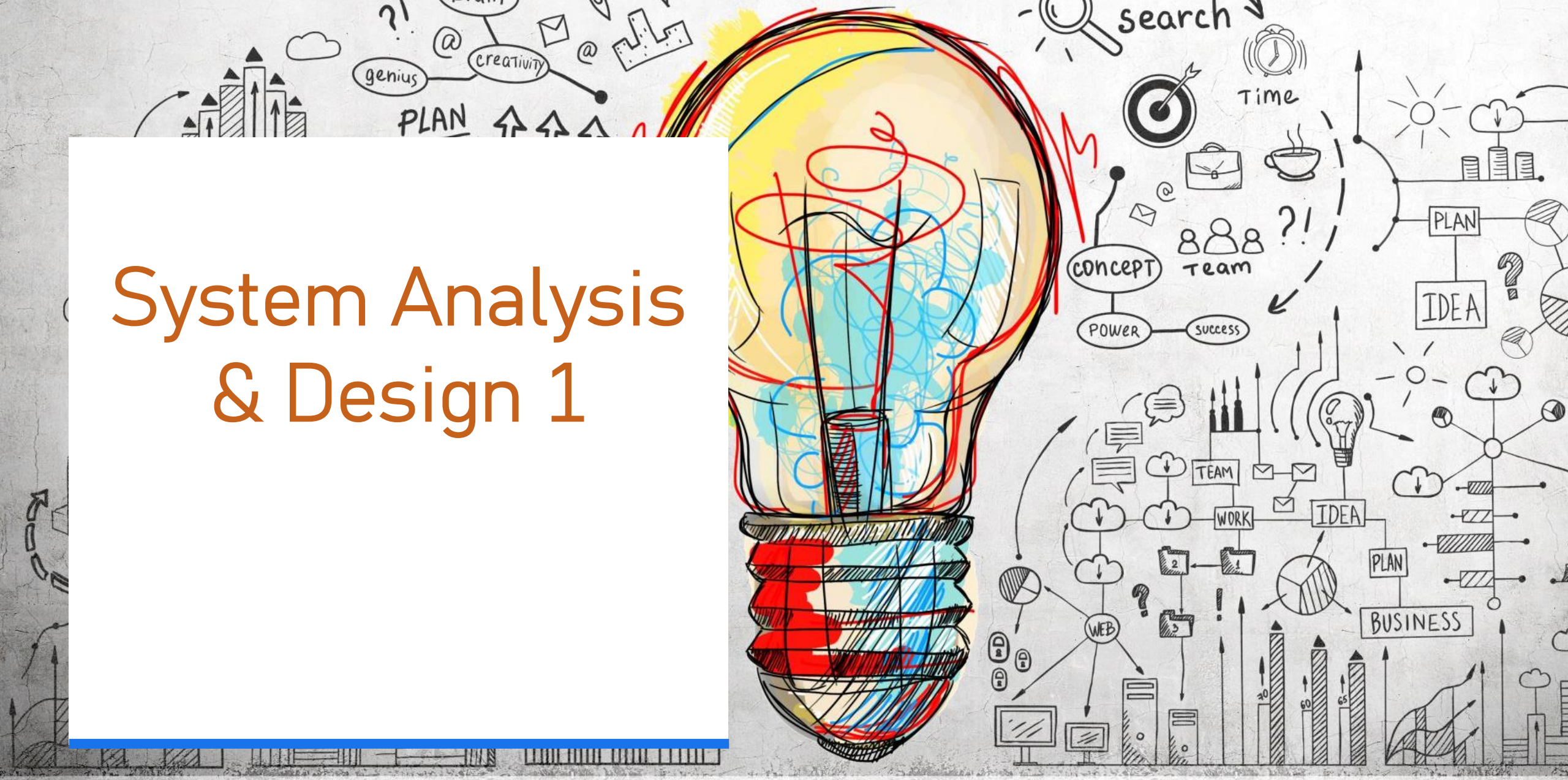
- Structured design methodologies adopt a formal step-by step approach to the SDLC that moves logically from one phase to the next.



See You Agai



System Analysis & Design 1



Agenda-Chapter 1 –Part 3



Systems Development Methodologies



Selecting The Best Systems Development Methodology



Roles In System Development Process

Systems Development Methodologies

- Systems Development Methodologies are structured approaches, models, or frameworks that guide the process of creating, designing, testing, and maintaining information systems or software.
- They provide a step-by-step roadmap for project teams, ensuring that development is organized, efficient, and aligned with business objectives.
- These methodologies define how tasks are divided, sequenced, and executed throughout the System Development Life Cycle (SDLC).
- Each methodology has its own strengths, limitations, and suitability depending on factors like project size, complexity, time constraints, and user involvement.

Systems Development Methodologies

SDM

Structured Design



Rapid Application Development (RAD)

Agile Development

Structured Design

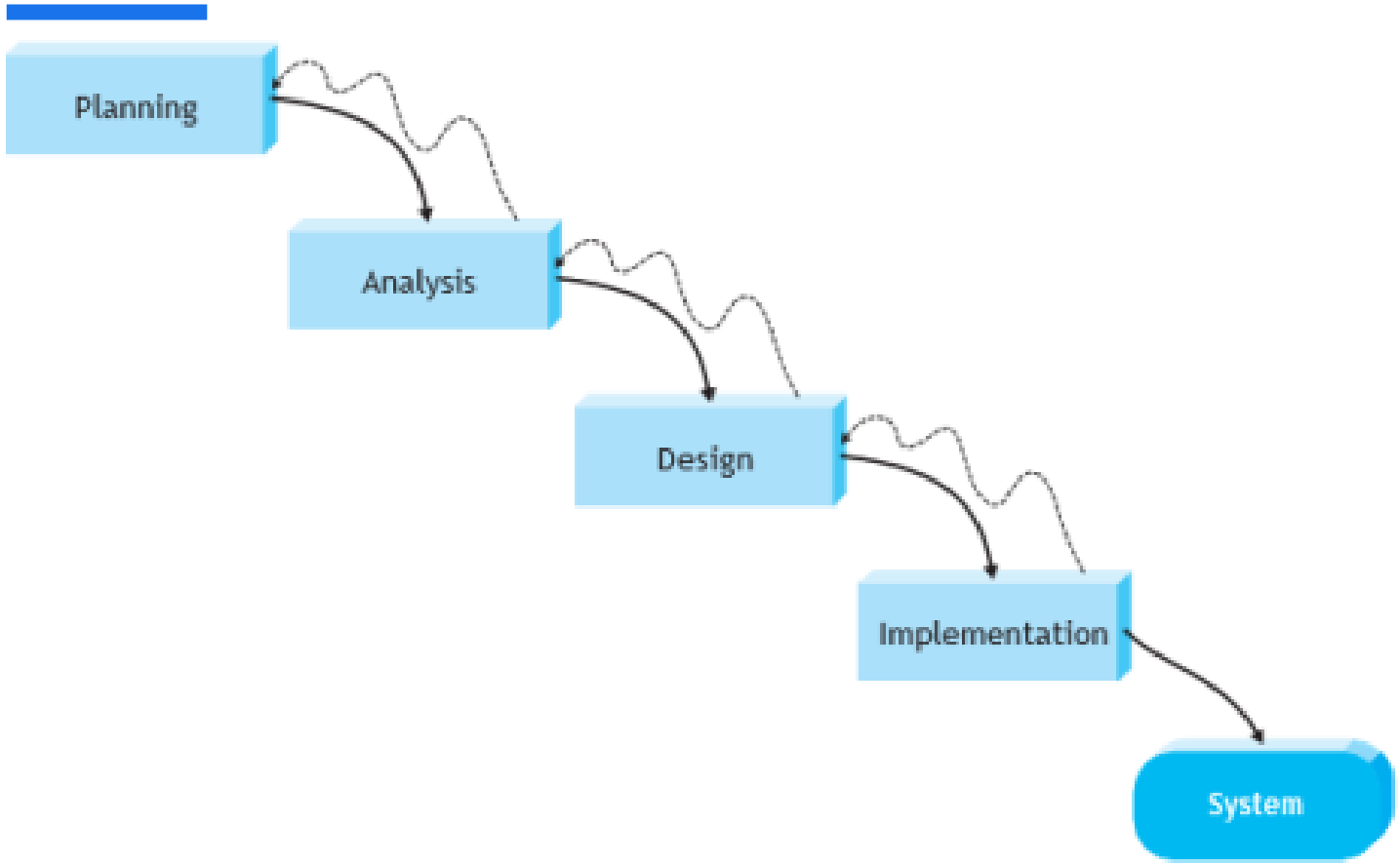
- Structured design methodologies adopt a formal step-by step approach to the SDLC that moves logically from one phase to the next.





Structured Design- Waterfall Development

- The original structured design methodology (still used today)
- With waterfall development–based methodologies, the analysts and users *proceed in sequence from one phase to the next* .
- The key deliverables for each phase are typically very long (often hundreds of pages in length) and are presented to the project sponsor for approval as the project moves from phase to phase.
-



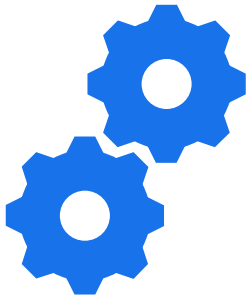
Structured Design- Waterfall Development

Structured Design- Waterfall Development

- This methodology is referred to as waterfall development because it *moves forward from phase to phase* in the same manner as a waterfall. Although it is possible to go backward in the SDLC (e.g., from design back to analysis)



Advantages of Waterfall Development



- *Clear and Well-Defined Phases*
- *Easy to Understand and Use*
- *Documentation*: emphasizes documentation at each stage of the process. This can be beneficial for future reference, maintenance
- *Early Detection of Issues*: Since each phase is completed before moving to the next, the structured design waterfall approach allows for early detection and mitigation of issues.

Waterfall Development Disadvantages

- *Lack of Flexibility:* The waterfall model is rigid and inflexible, as it follows a *sequential flow*. Once a phase is completed, it is challenging to make changes without disrupting subsequent phases.

- *Late Delivery of Working Software:* Since the structured design waterfall approach follows a sequential process, *working software* is typically *delivered at the end of the development cycle.* This can lead to delays in providing tangible results and *limit opportunities for early user feedback.*

Waterfall Development Disadvantages

- *Risk of Requirement Misinterpretation:* With the waterfall model, requirements are typically gathered and frozen at the beginning. This can lead to a higher risk of requirement misinterpretation or misunderstanding, as changes are more difficult to accommodate later in the process.

- *Limited Adaptability to Changes:* This can be problematic in *dynamic environments* where requirements evolve, or new insights emerge during the development process. *Adapting to changes becomes challenging, potentially resulting in an outdated or less valuable* final product.

Parallel Development



Parallel Development methodology attempts to address the problem of long delays between the **analysis** phase and **the delivery of the system**.

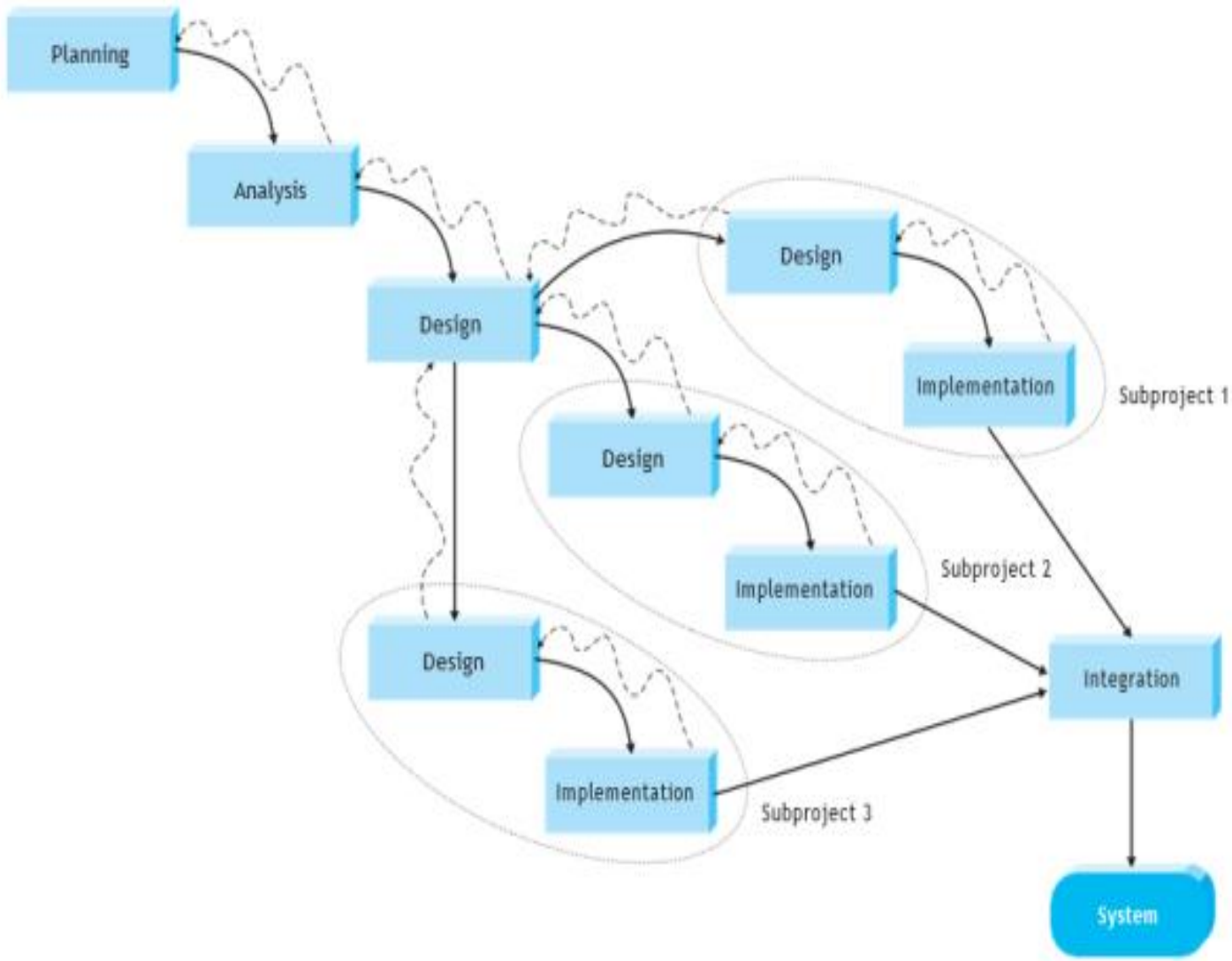


Instead of doing design and implementation in sequence, it performs a general design for the whole system and then divides the project into a series of distinct subprojects that can be **designed and implemented in parallel.**

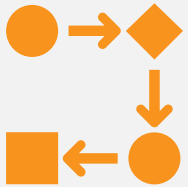


Once all subprojects are complete, there is a **final integration** of the separate pieces, and the system is delivered.

Parallel Development



Parallel Development

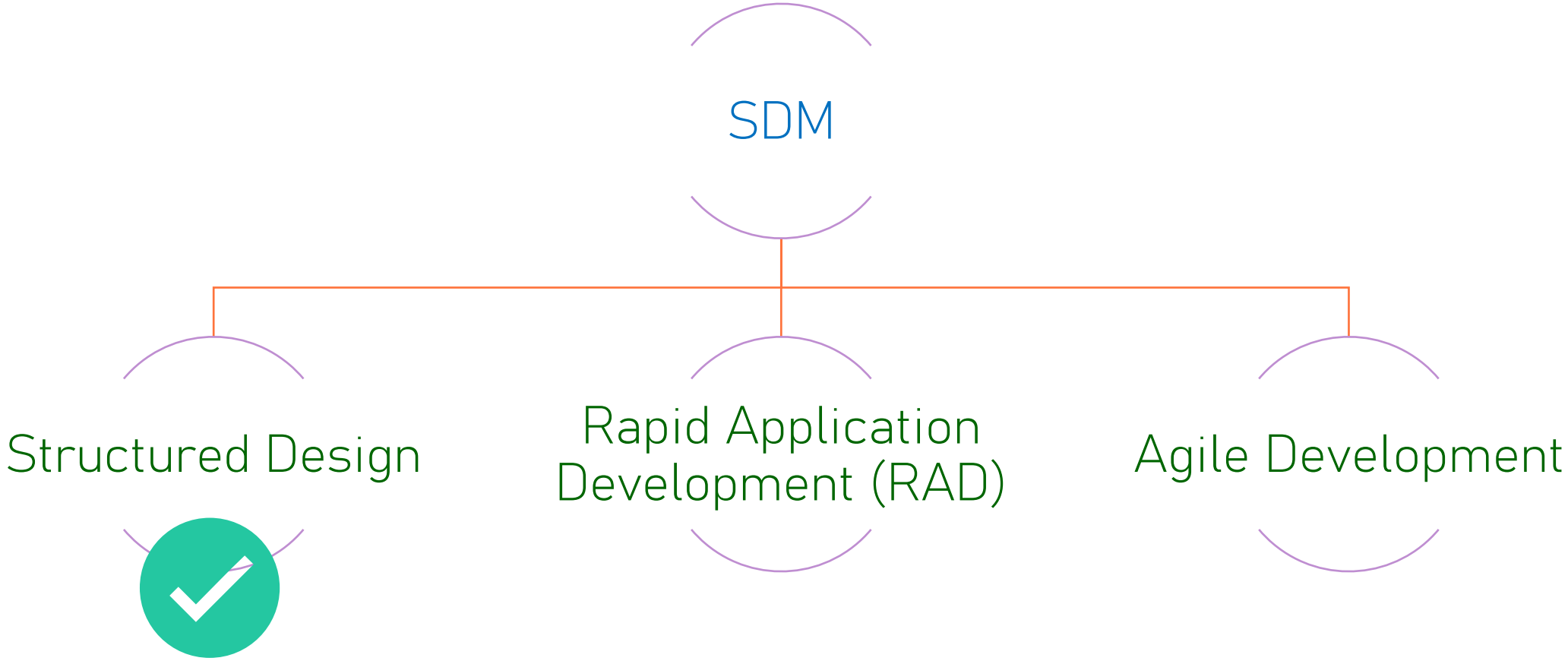


The primary advantage of this methodology is that it can reduce the schedule time to deliver a system; thus, there is less chance of changes in the business environment causing rework. However, the approach still suffers from problems caused by paper documents.



It also adds a new problem: Sometimes the subprojects are not completely independent; design decisions made in one subproject may affect another, and the end of the project may require significant integration efforts.

Systems Development Methodologies



Rapid Application Development (RAD)



These are a newer class of systems development methodologies that emerged in the 1990s.



RAD-based methodologies attempt to address *weaknesses of structured design* methodologies

By adjusting the SDLC phases to get some *part of the system developed quickly* and into the hands of the users.



In this way, the users can better understand the system and suggest revisions that bring the system closer to what is needed

Rapid Application Development (RAD)

- Most RAD-based methodologies recommend that analysts use *special techniques* and *computer tools* to speed up the analysis, design, and implementation phases, such as CASE tools, joint application Development (JAD) sessions, fourth-generation programming languages that simplify and speed up programming (e.g., **Perl, Python, Ruby, SQL**), and code generators that automatically produce programs from design specifications.

Rapid Application Development (RAD)



The combination of the changed SDLC phases and the use of these ***tools*** and techniques improves the *speed and quality of systems development.*

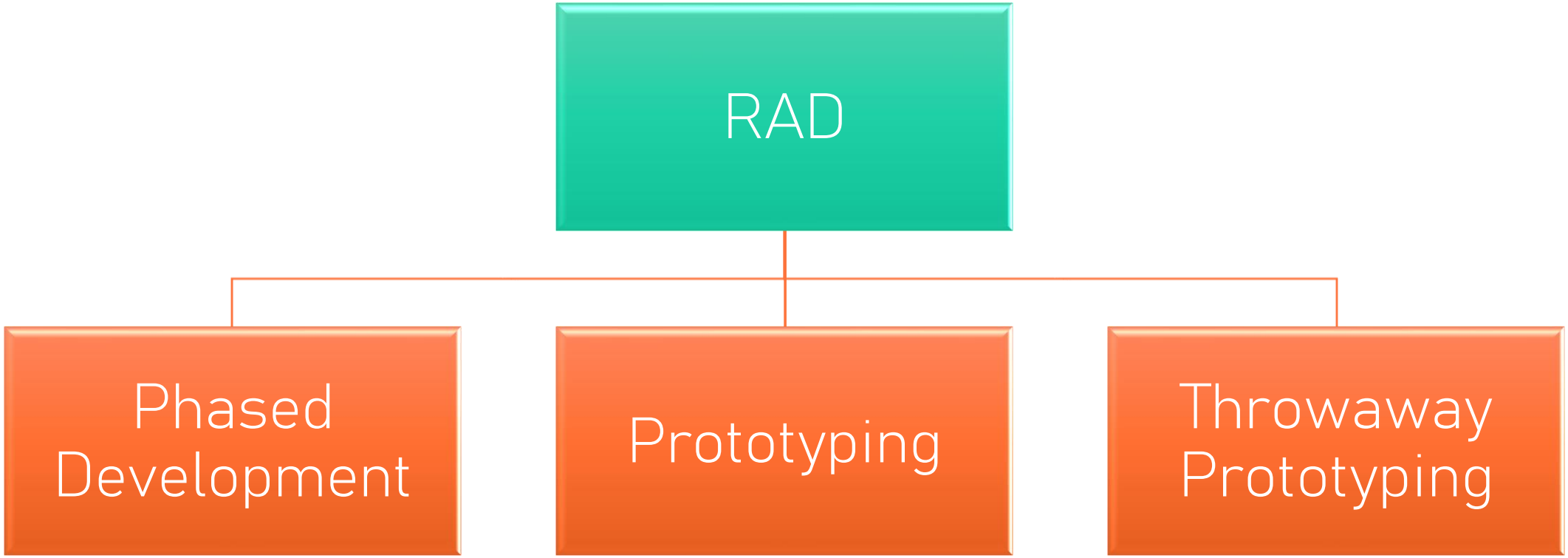


However, there is one possible problem with RAD-based methodologies: *managing user expectations*.



As a user better understands the information technology, the systems *requirements tend to expand*.

Rapid Application Development (RAD)



Phased Development



A phased development–based methodology **breaks an overall system into a series of versions**, which are developed sequentially.

The analysis phase identifies the overall system ***concept, and the project team, users, and system sponsor*** then categorize the *requirements into a series of versions*.



The most important and *fundamental requirements* are bundled into the ***first version of the system.***

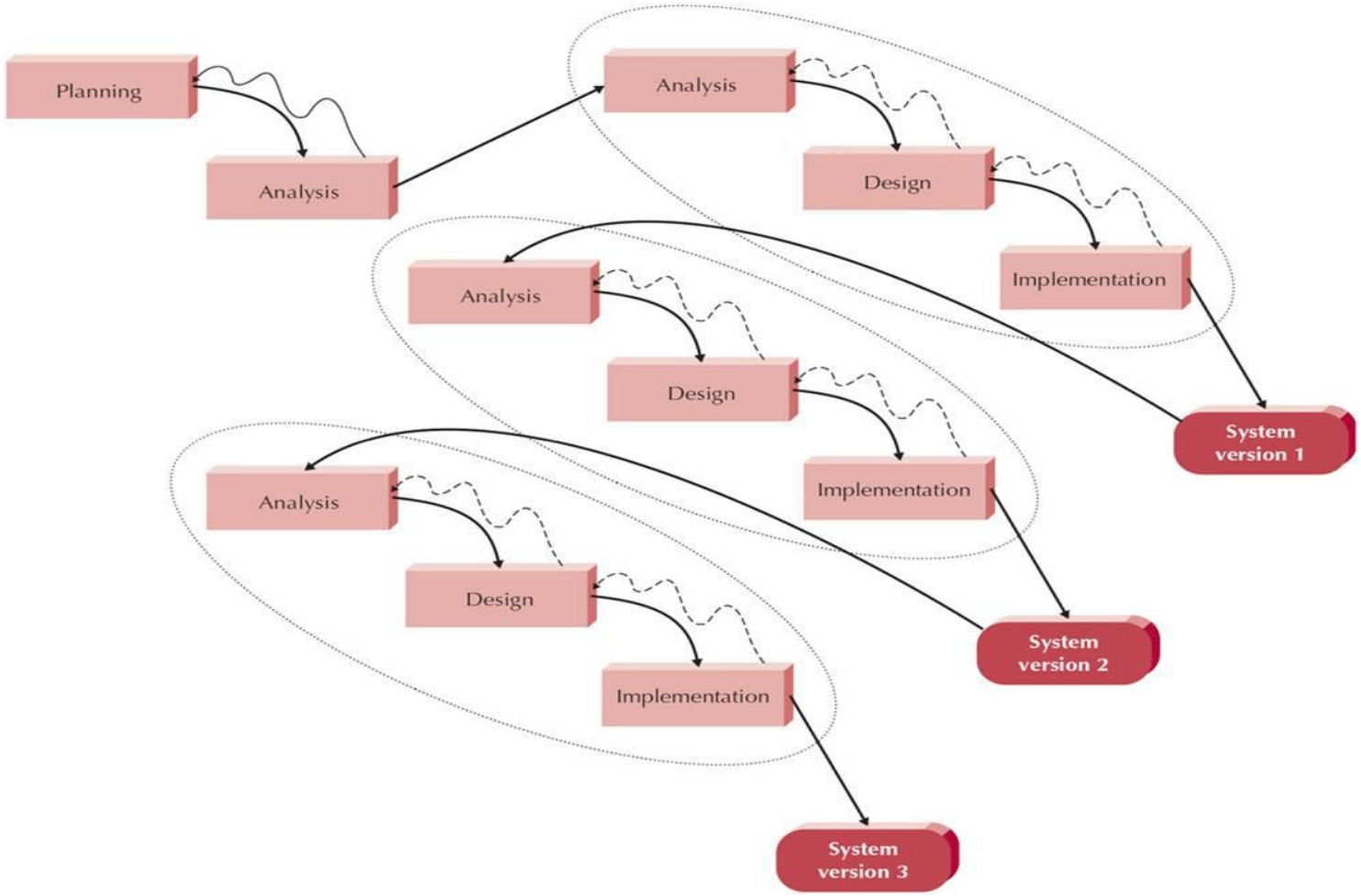


The analysis phase then leads into design and implementation but only with the set of requirements identified for version 1.



Phased Development

- Once version 1 is implemented, work begins on *version 2*.
- *Additional analysis is performed based on the previously identified requirements and combined with new ideas* issues that arose from the *users' experience with version 1*.
- Version 2 then is designed and implemented and *work immediately begins on the next version*.
- This process continues *until the system is complete* or is no longer in use.



Phased Development



Phased Development pros

- Phased development–based methodologies have the advantage of *quickly getting a useful system into the hands of the users*.
- Although the system does not perform all the functions the users need at **fi**rst, it does begin to provide *business value sooner* than if the system were delivered after completion
- Likewise, because users begin to work with the system sooner, they are more likely to *identify important additional requirements* sooner than with structured design situations

Phased Development Cons



The major drawback to phased development is that users begin to work with systems that are intentionally incomplete.

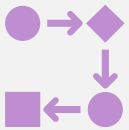


It is critical to identify the most important and useful features and include them in the first version and to **manage users' expectations** along the way.

Prototyping



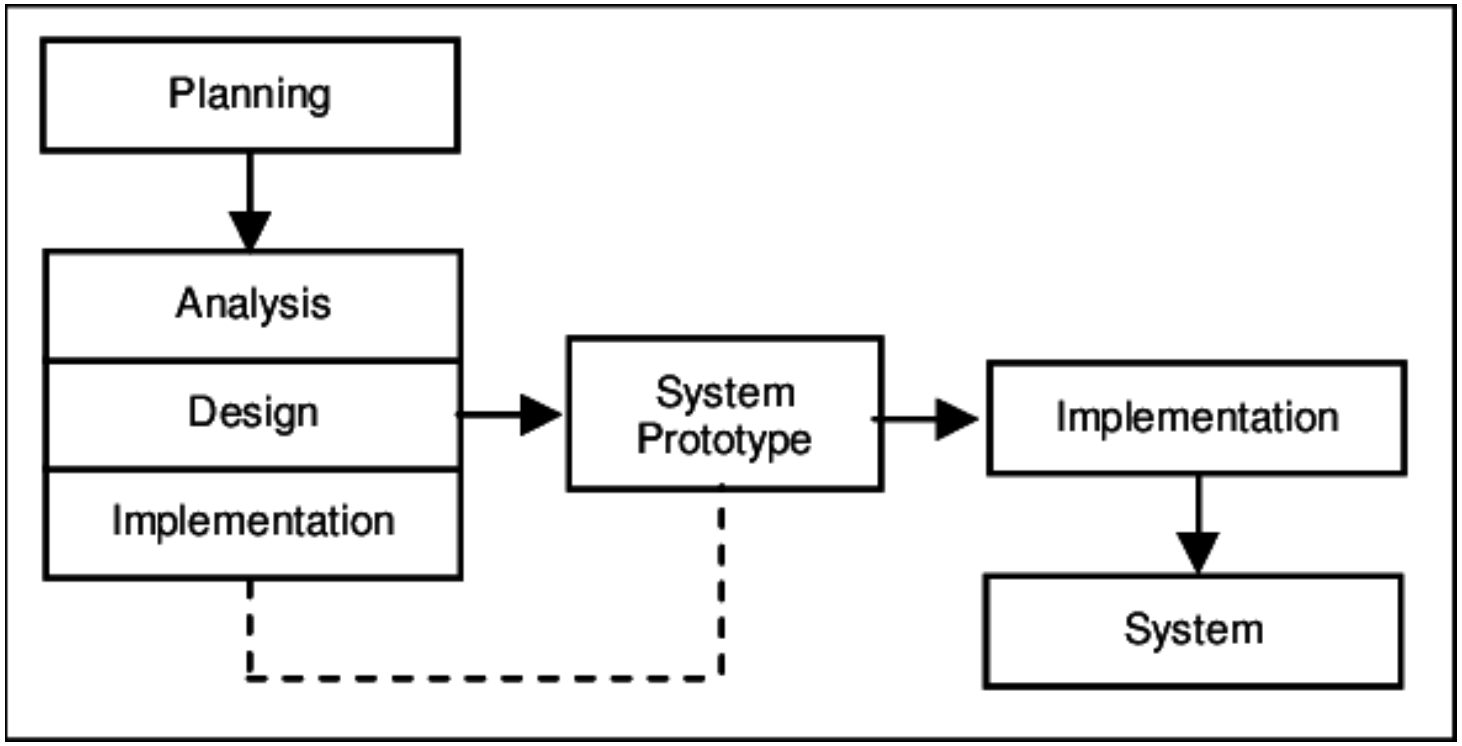
A prototyping-based methodology performs the *analysis, design, and implementation phases concurrently*, and all *three phases are performed repeatedly in a cycle until the system is completed*.



With these methodologies, the basics of analysis and design are performed, and work immediately begins on ***a system prototype***, a "quick-and-dirty" program that provides a minimal number of features.

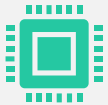
Prototyping

- The ***first prototype is usually the first part of the system*** that is used. This is ***shown to the users who provide comments***.
- These comments are used to ***reanalyze, redesign, and reimplement a second prototype***, which provides a few more features.
- This process continues in a cycle until the analysts, users, and sponsor agree that the ***prototype provides enough functionality*** to be installed and used in the organization.
- After the prototype (now called the system) is installed, refinement occurs until it is accepted as the new system.



Prototyping

Prototyping Pros



The key advantage of a prototyping-based methodology is that it *very quickly provides a system* with which the users can interact, even if it is not ready for widespread organizational use at first.



Prototyping reassures the users that the project team is working on the system (*there are no long delays* in which the users see little progress),



prototyping helps to more quickly *refine real requirements*



Rather than attempting to understand a system specification on paper, the users can interact with the *prototype to better understand what it can and cannot do*

Prototyping Cons



its fast-paced system releases challenge attempts to conduct **careful, methodical analysis.**



Often the prototype undergoes such significant changes that **many initial design decisions become poor ones.**

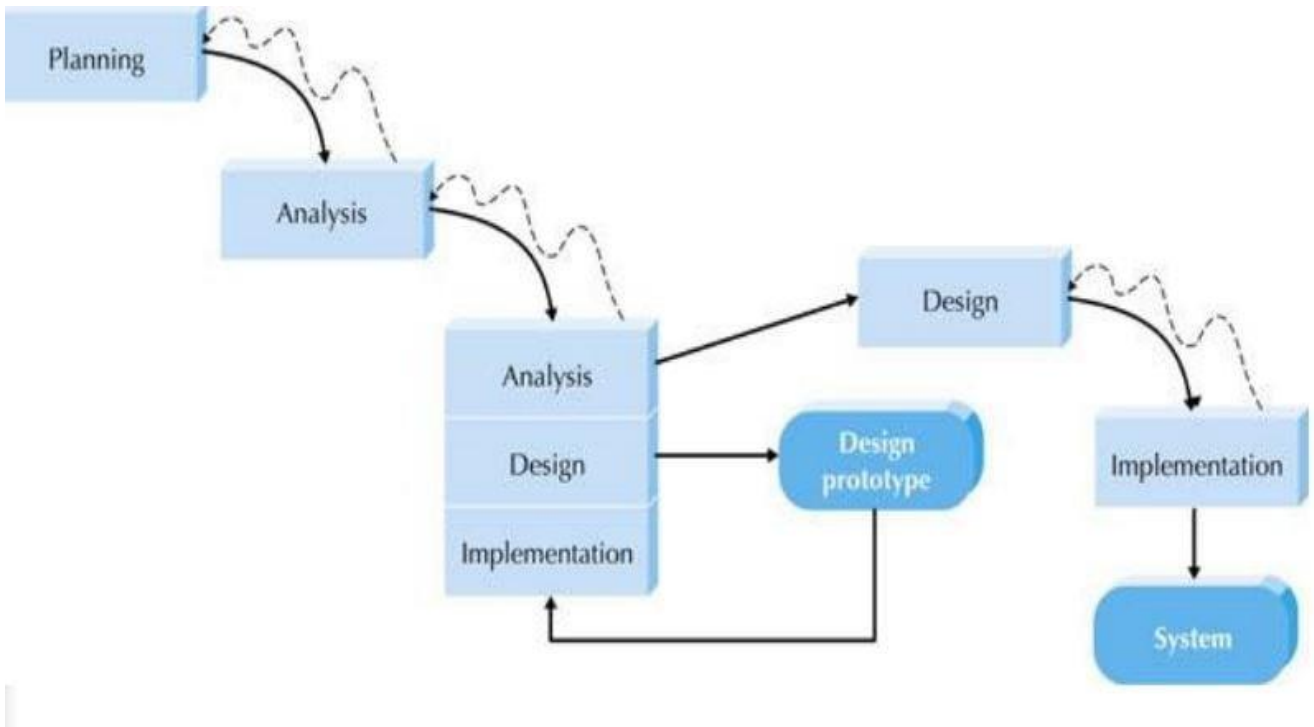


This can cause **problems in the development of complex systems** because fundamental issues and problems are not recognized until well into the development process.

Throwaway Prototyping

Throwaway prototyping–based methodologies are like prototyping-based methodologies in that they include the development of prototypes however, throwaway prototypes are done at a different point in the SDLC.

These prototypes are used for a very different purpose than those previously discussed, and they have a very different appearance.



Throwaway Prototyping

Throwaway Prototyping



A system developed using this type of methodology probably relies on several design prototypes during the analysis and design phases.



Each of the prototypes is used to minimize the risk associated with the system by confirming that important issues are understood before the real system is built. Once the issues are resolved, the project moves into design and implementation.



At this point, the design prototypes are thrown away, which is an important difference between these methodologies and prototyping methodologies, in which the prototypes evolve into the final system.

Throwaway Prototyping

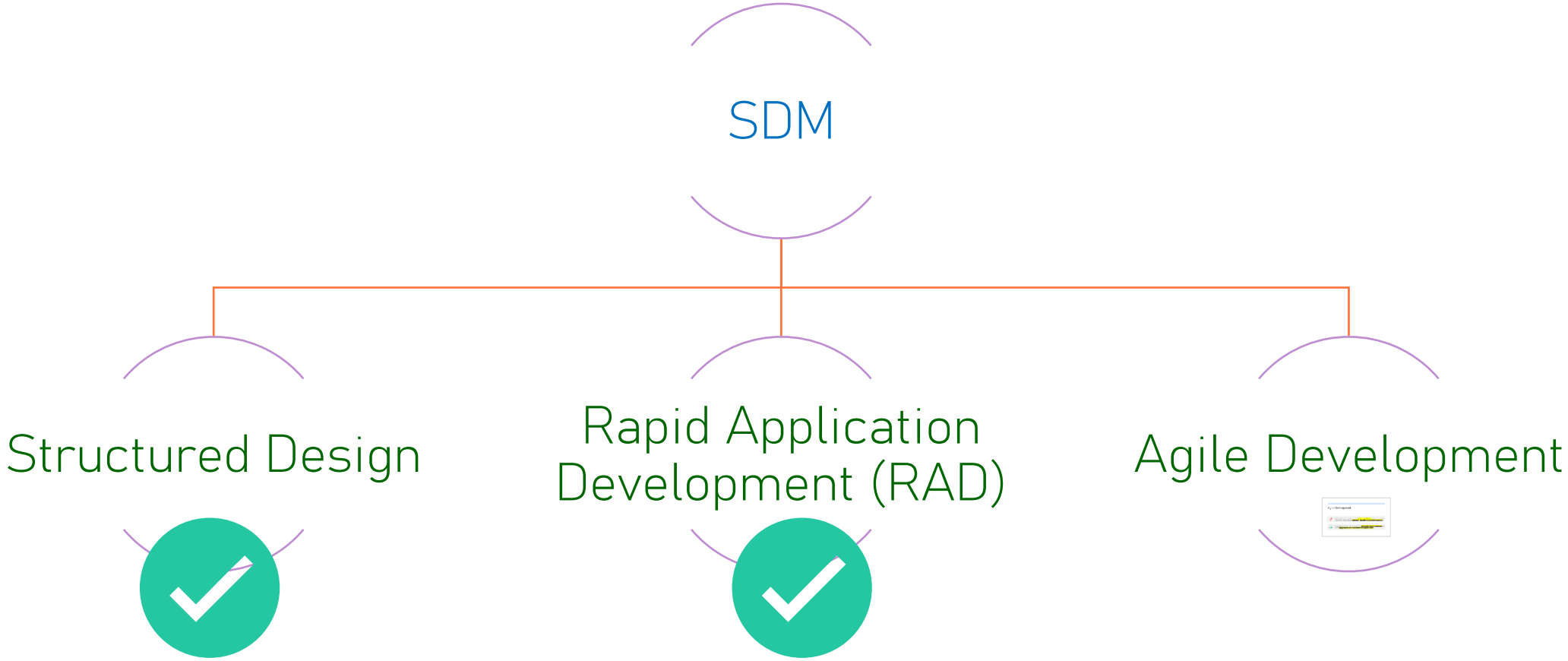


Throwaway prototyping-based methodologies balance the benefits of well-thought-out **analysis and design** phases with **the advantages of using prototypes** to **refine key issues before a system is built**.



It may take longer to deliver the final system as compared to **prototyping-based methodologies** (because the prototypes do not become the final system), but this type of methodology usually produces more stable and reliable systems

Systems Development Methodologies



Agile Development



Agile development approach is an iterative and flexible approach to software development that emphasizes **collaboration, adaptability, and continuous improvement**

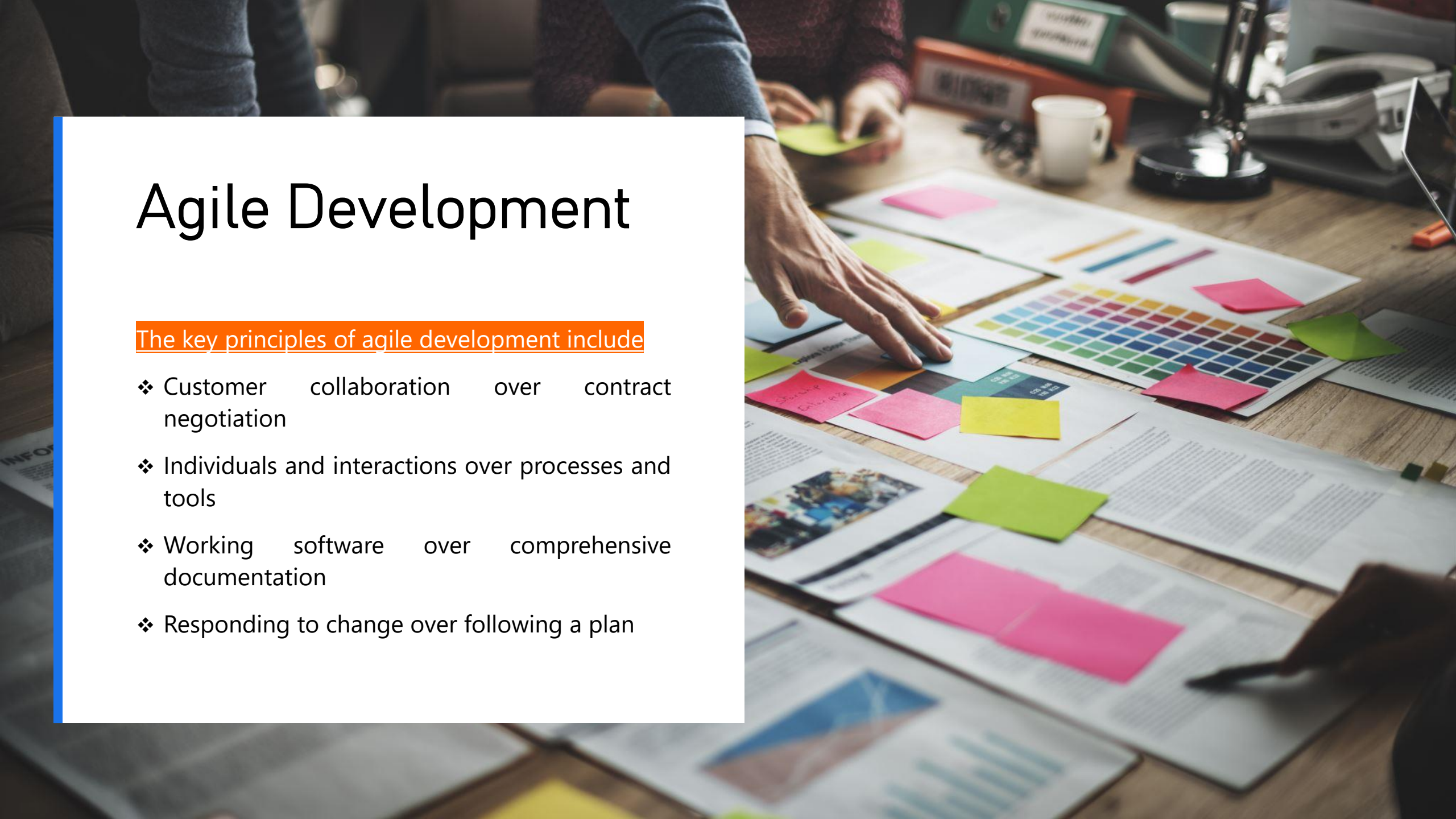


It has gained popularity due to its ability to **accommodate changing requirements** and deliver **high-quality software in a more efficient and adaptable manner.**

Agile Development

The key principles of agile development include

- Customer collaboration over contract negotiation
- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Responding to change over following a plan



Agile Development

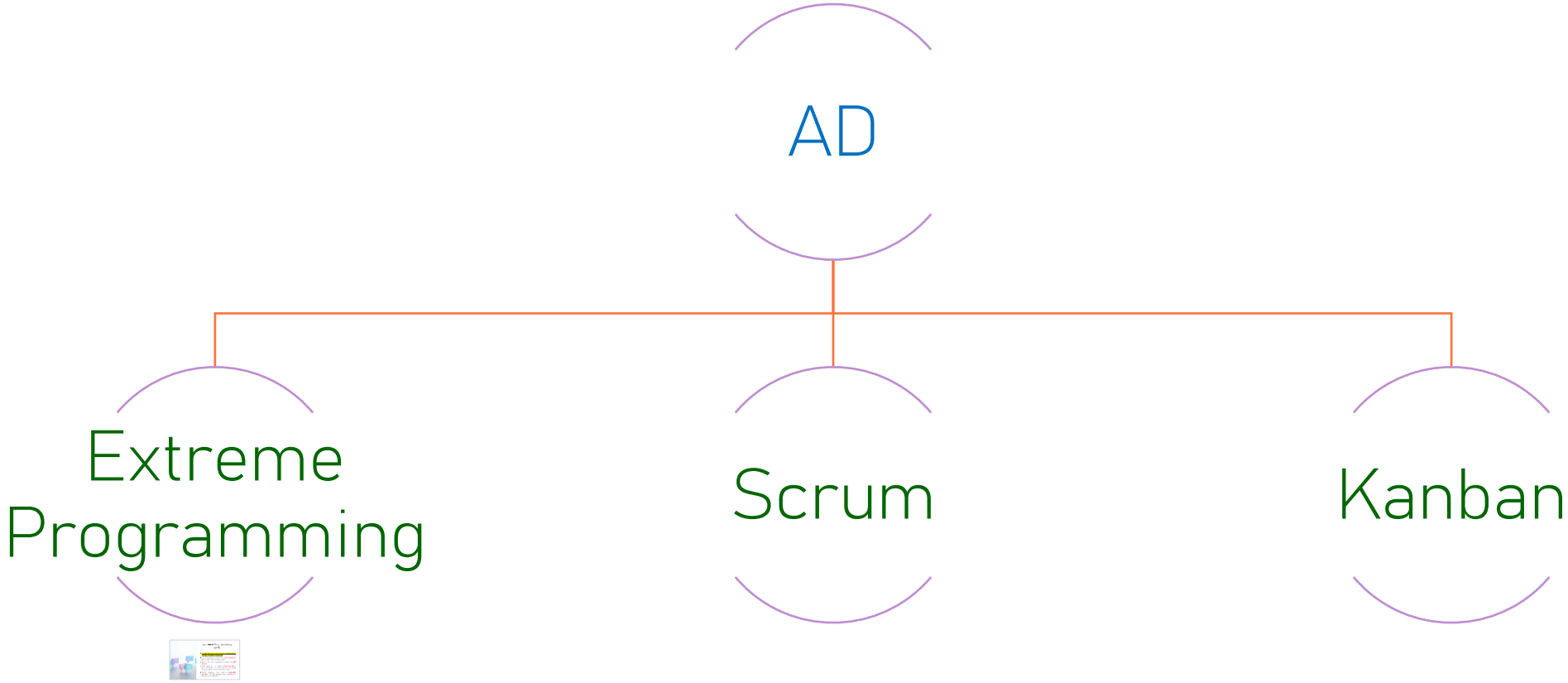
- They typically involve breaking down the **project into small, manageable increments called "sprints" or "iterations,"** with each iteration delivering a working piece of software.

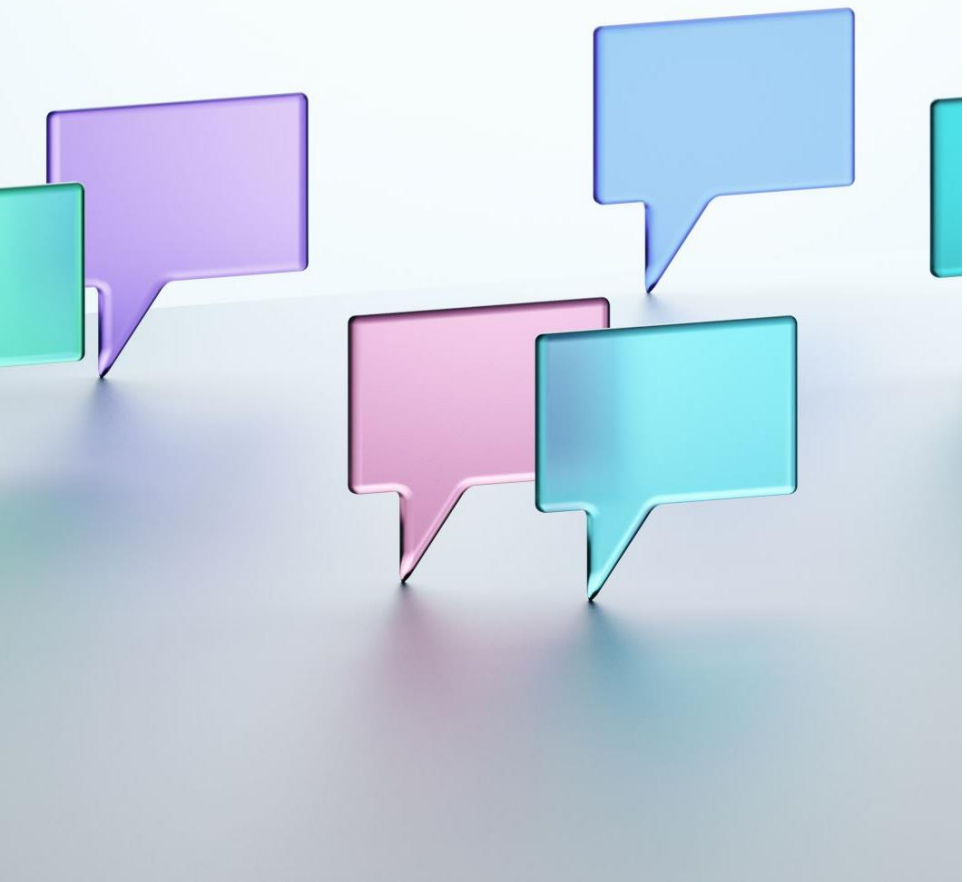
- The development team holds **regular meetings, such as daily stand-ups, to discuss progress**, identify and address any roadblocks, and plan the next steps.

- **Continuous feedback and adaptation** are fundamental to agile methodologies. At the end of each iteration, the team reviews the progress, gathers feedback, **and incorporates it into the subsequent iterations.**

- This iterative approach allows for early and frequent **delivery of working software, increasing customer satisfaction, and reducing the risk** of large-scale failures.

Agile Development





Extreme Programming (XP)

- Founded on four core values: ***communication, simplicity, feedback, and courage.***
- First, the developers must provide rapid feedback to the end users on a continuous basis.
- Second, XP requires developers to follow the KISS principle.
- Third, developers must make incremental changes to grow the system, and they must not only accept change, but they must also embrace change.
- Fourth, developers must have a quality-**fi**rst mentality. XP also supports team members in developing their own skills

Principles of (XP)

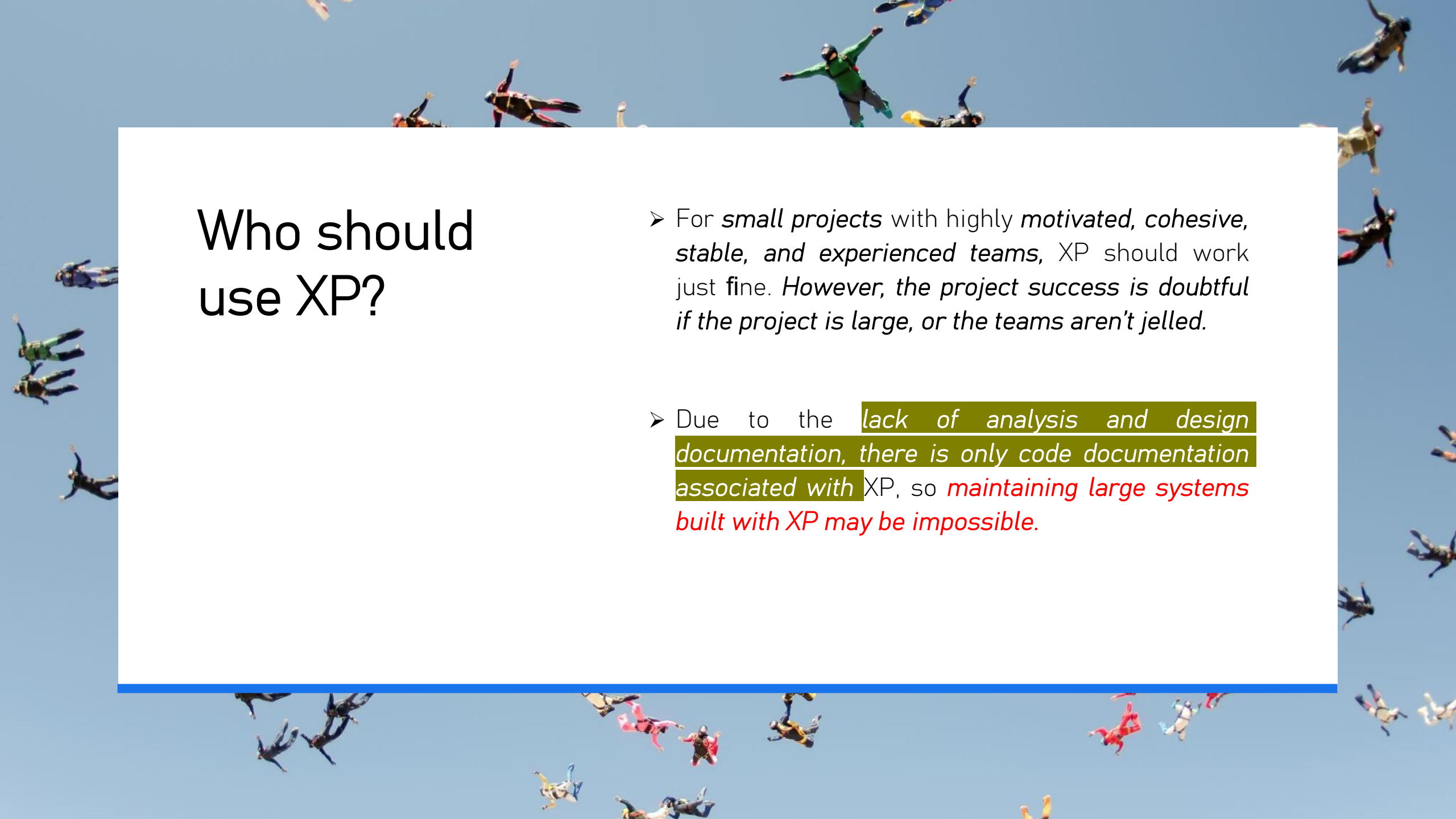
- Three of the key principles that XP uses to create successful systems are continuous testing, simple coding performed by pairs of developers, and close interactions with end users to build systems very quickly.

- After a superficial planning process, projects perform analysis, design, and implementation phases iteratively.

Core values of (XP)

- Testing and efficient coding practices are core to XP. In fact, code is tested each day and is placed into an integrative testing environment. If bugs exist, the code is backed out until it is completely free of errors.

- XP relies heavily on refactoring, which is a disciplined way to restructure code to keep it simple.



Who should use XP?

- For *small projects* with highly *motivated, cohesive, stable, and experienced teams,* XP should work just fine. *However, the project success is doubtful if the project is large, or the teams aren't jelled.*

- Due to the *lack of analysis and design documentation, there is only code documentation associated with* XP, so *maintaining large systems built with XP may be impossible.*



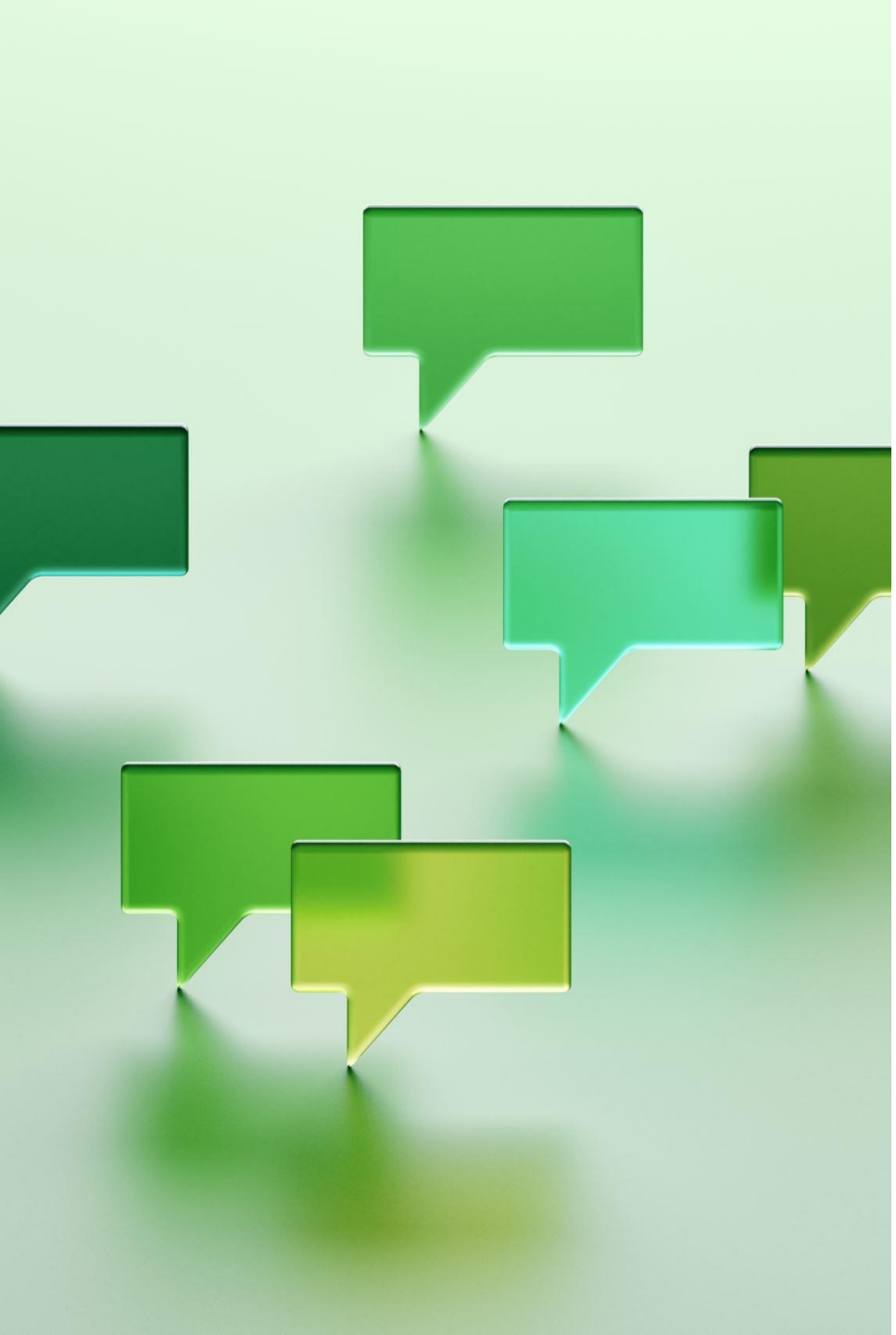
Selecting the Appropriate Development Methodology

- Because there are many methodologies, the **fi**rst challenge faced by analysts is to select which methodology to use.

- Choosing a methodology is not simple, because no one methodology is always best. (If it were, we'd simply use it everywhere!) Many organizations have standards and policies to guide the choice of methodology.

Selecting the Appropriate Development Methodology

- Clarity of User Requirements
- Familiarity with Technology
- System Complexity
- System Reliability
- Short Time Schedules
- Schedule Visibility

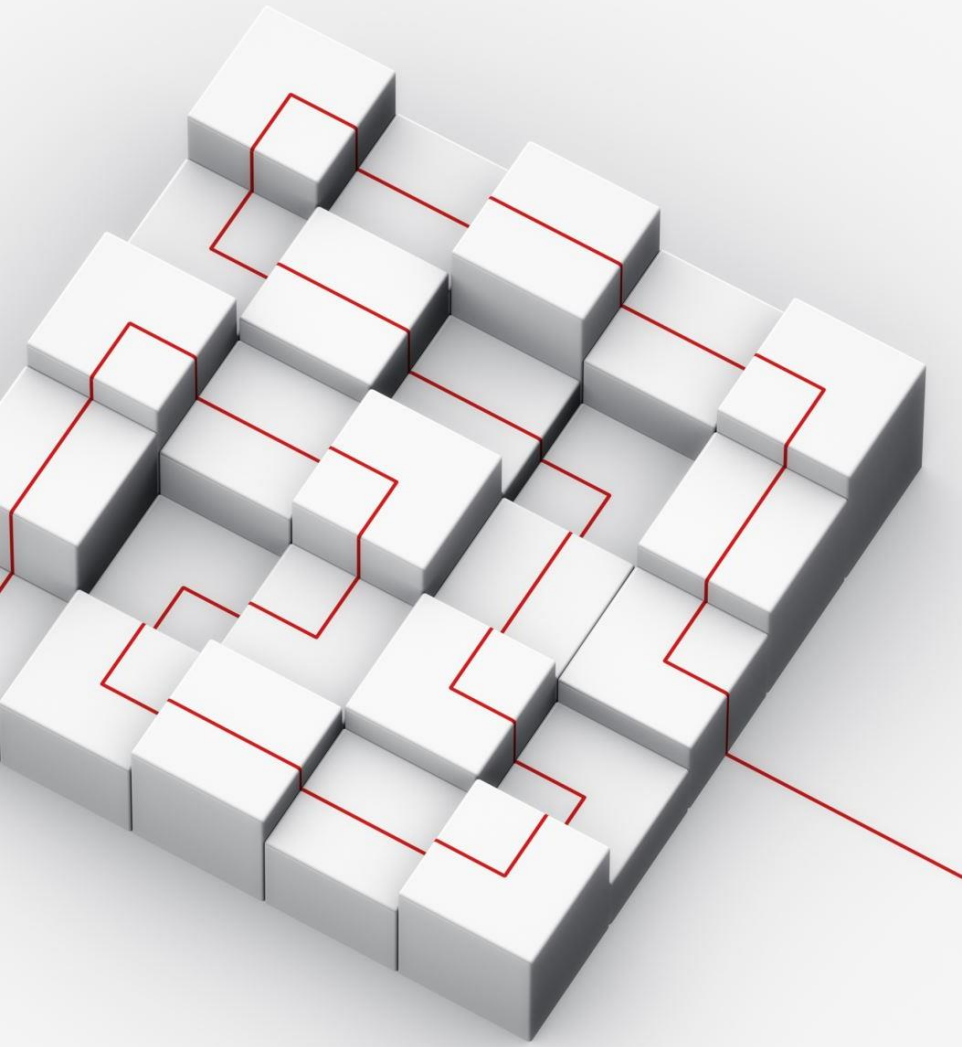


Selecting the Appropriate Development Methodology

- **Clarity of User Requirements** When the user requirements for a system are unclear, it is difficult to understand them by talking.
- ***Agile approach*** is perfect when requirements are unclear
- ***Prototyping- and throwaway prototyping–based RAD methodologies*** are usually more appropriate when user requirements are unclear because they provide prototypes for users to interact with early in the SDLC.

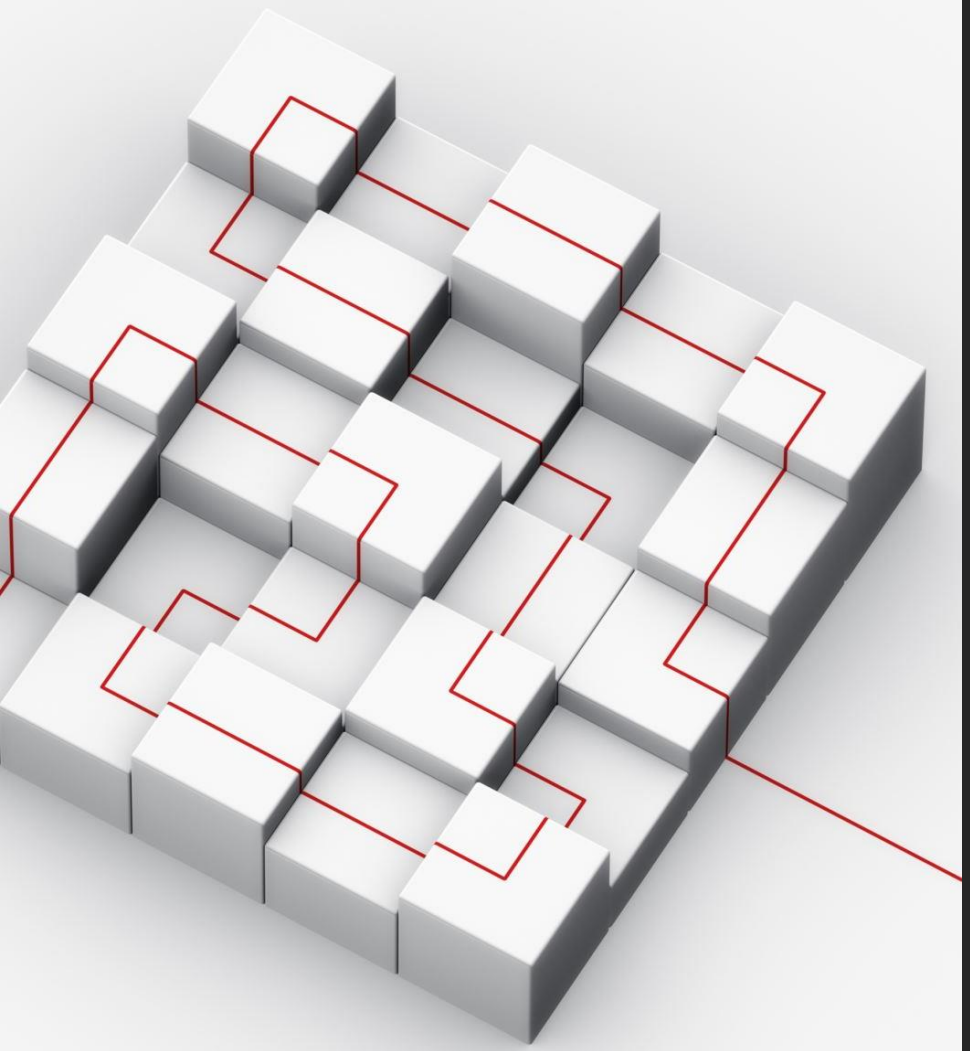
Selecting the Appropriate Development Methodology

- **Familiarity with Technology** When the system will use new technology with which the analysts and programmers are not familiar (e.g., the first Web development project with Java), risks increase because the tools might not be capable of doing what is needed.
- ***Agile works well with new technology.***
- ***Throwaway prototyping–based methodologies*** are particularly appropriate if users lack familiarity with technology because they explicitly encourage the developers to develop design prototypes for areas with high risks.
- ***Phased development–based methodologies*** are good as well, because they create opportunities to investigate the technology in some depth before the design is complete.
- Although you might think **prototyping-based methodologies** are also appropriate, they are much **less appropriate** because the early prototypes that are built usually only scratch the surface of the new technology.



Selecting the Appropriate Development Methodology

- **System Complexity** require careful and detailed analysis and design.
- Agile works very well with complex system
- ***Throwaway prototyping–based methodologies*** are particularly well suited to such detailed analysis and design, as opposed to prototyping-based methodologies, which are not.



Selecting the Appropriate Development Methodology

- The ***traditional structured design–based methodologies*** can handle complex systems, but without the ability to get the system or prototypes into the users' hands early on, some key issues may be overlooked.

Selecting the Appropriate Development Methodology

- System Reliability is usually an important factor in system development after all, who wants an unreliable system?
- *Agile and Throwaway prototyping methodologies* are the most appropriate for reliable systems
- Prototyping methodologies are generally not a good choice when reliability is critical because it lacks the careful analysis and design phases that are essential for dependable systems





Selecting the Appropriate Development Methodology

- **Short Time Schedules** Projects that have short time schedules are well suited with *Agile and RAD- based methodologies*. This is due to them being designed to increase the speed of development.
- Waterfall-based methodologies are the worst choice when time is at a premium because they do not allow for easy schedule changes.





Selecting the Appropriate Development Methodology

- **Schedule Visibility** One of the greatest challenges in systems development is determining whether a project is on schedule.
- The ***structured design*** methodologies fits well in this case due to the plan that follows.
- RAD and Agile may not be suitable to follow a fixed plan.

Roles in System Development Process



- Business Analyst
- System Analyst
- Infrastructure Analyst
- A Change Management Analyst
- Project Manager

Roles In System Development Process



- ✓The project team needs a variety of skills. All analysts need to have general skills, such as *change management, ethics, communications, and technical*.

However, different kinds of analysts require specific skills in addition to these.

- ✓*Business analysts* usually have business skills that help them to understand the business issues surrounding the system,
- ✓whereas *systems analysts* also have significant experience in analysis and design and programming.

Roles in System Development Process



- ✓The *infrastructure analyst* focuses on technical issues surrounding how the system will interact with the organization's technical infrastructure.
- ✓The *change management analyst* focuses on people and management, issues surrounding the system installation.
- ✓In addition to analysts, project teams will include a *project manager, Technical Writers, programmers, and other specialists*.

Business Analyst

A business analyst focuses on the business issues surrounding the system.

1-These issues include identifying the business value that the system will create,

2-developing ideas and suggestions for how the business processes can be improved,

3-and designing the new processes and policies in conjunction with the systems analyst.

Systems Analyst

- A systems analyst focuses on the IS issues surrounding the system.
- This person develops ideas and suggestions for how information technology can improve business processes, designs the new business processes with help from the business analyst,
- Designs the new information system and ensures that all IS standards are maintained.
- A systems analyst will likely have significant training and experience in analysis and design, programming, and even areas of the business.

Infrastructure Analyst

- An infrastructure analyst focuses on the ***technical issues*** surrounding ***how the system will interact with the organization's technical infrastructure*** (e.g., hardware, software, networks, and databases).
- This individual will probably have significant training and ***experience in networking, database administration, and various hardware and software*** products.
- An infrastructure analyst's tasks include ensuring that the ***new information system conforms to organizational standards and identifying infrastructure changes*** needed to support the system.



A Change Management Analyst

- A change management analyst focuses on the ***people and management issues*** surrounding the system installation.

- The roles of this person include ensuring that the ***adequate documentation and support are available to users***, providing user ***training on the new system***, and developing strategies to ***overcome resistance*** to change.

Project Manager

- A project manager is responsible for ensuring that the project is completed on time and within budget and that the system delivers all benefits intended by the project sponsor.

- The role of the project manager includes managing the team members, developing the project plan, assigning resources, and being the primary point of contact when people outside the team have questions about the project.

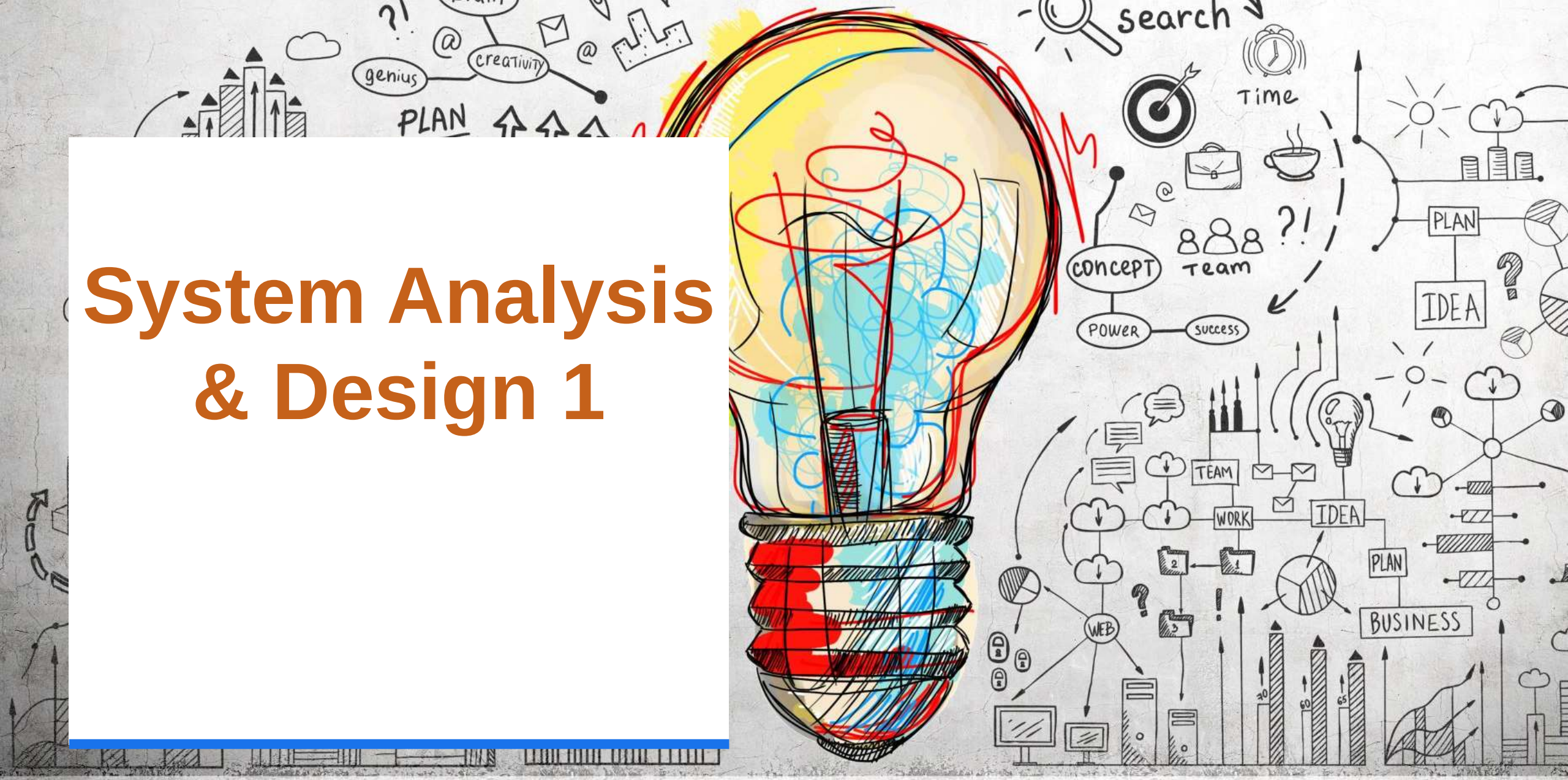
- This individual will likely have significant experience in project management and has probably worked for many years as a systems analyst beforehand.



See You Agai

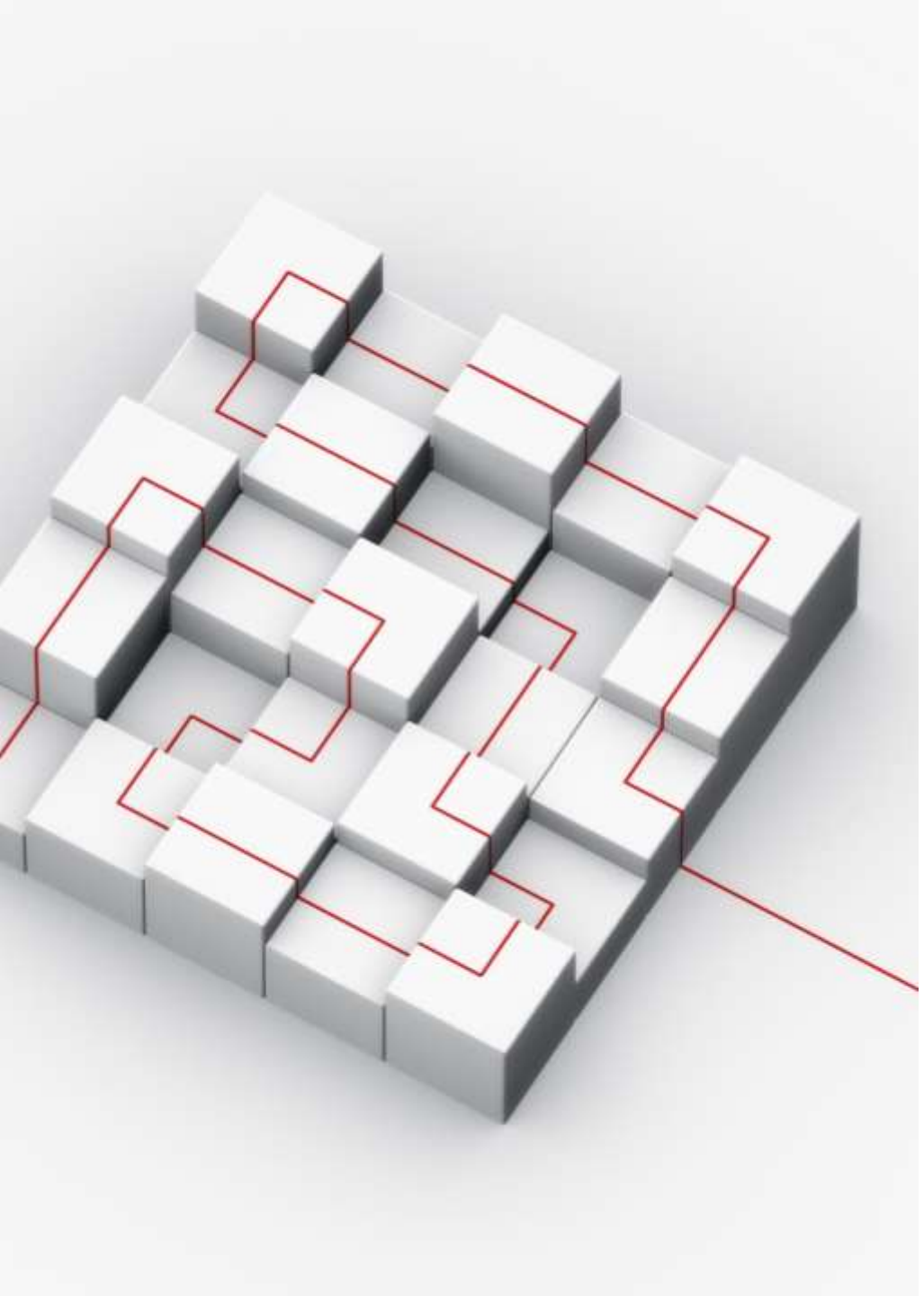


System Analysis & Design 1



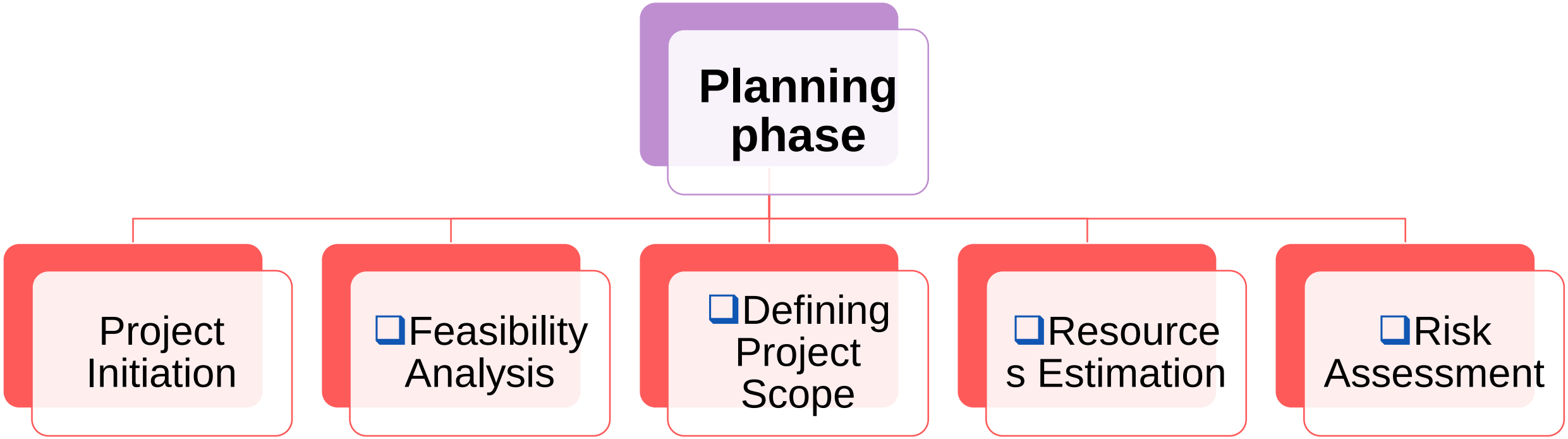
Agenda
Chapter 2
Part 1

- Project Initiation
- Feasibility Analysis
- Defining Project Scope
- Resources Estimation



Planning Phase

- The planning phase is the foundational stage of the System Development Life Cycle (SDLC).
- It sets the direction for the entire project and determines whether the proposed system is worth pursuing.
- This phase answer Question like, why we need a system? Does this system worth development?


Planning phase
Project Initiation
❑Feasibility Analysis
❑Defining Project Scope
❑Resource s Estimation
❑Risk Assessment

A-Project Initiation



Project initiation marks the very first step in the planning phase of the System Development Life Cycle (SDLC).



At this stage, a problem, opportunity, or business need is formally recognized and proposed as a project idea.



The goal is to establish ***why*** the project should exist and to ensure that it aligns with the strategic objectives of the organization.

A-Project Initiation

Recognizing	Recognizing a Business Need
Identifying	Identifying Opportunities for Improvement
Submitting	Submitting a System Request

1-Recognizing a Business Need

Starting Point: System development begins with recognizing a business need.

Sources of Need: Needs arise from inefficiencies, outdated technology, customer demands, or regulatory pressures.

Examples: Issues like declining sales or system downtime can signal the need for change.

Evidence of Need: Metrics, audit reports, and customer or employee feedback often reveal these needs.

Opportunities: Recognition also includes identifying growth opportunities, not just fixing problems (e.g., adopting e-commerce).

1-Recognizing a Business Need

Process: Recognition often starts informally through employee concerns, customer complaints, or management observations.

Documentation: Needs should be clearly defined and measurable (e.g., “reduce order processing time by 30%”).

Prioritization: Not every issue requires action—organizations must weigh costs, benefits, and strategic importance.

Decision-Making: Leaders assess whether the need justifies developing a new system.

2-Identifying Opportunities for Improvement

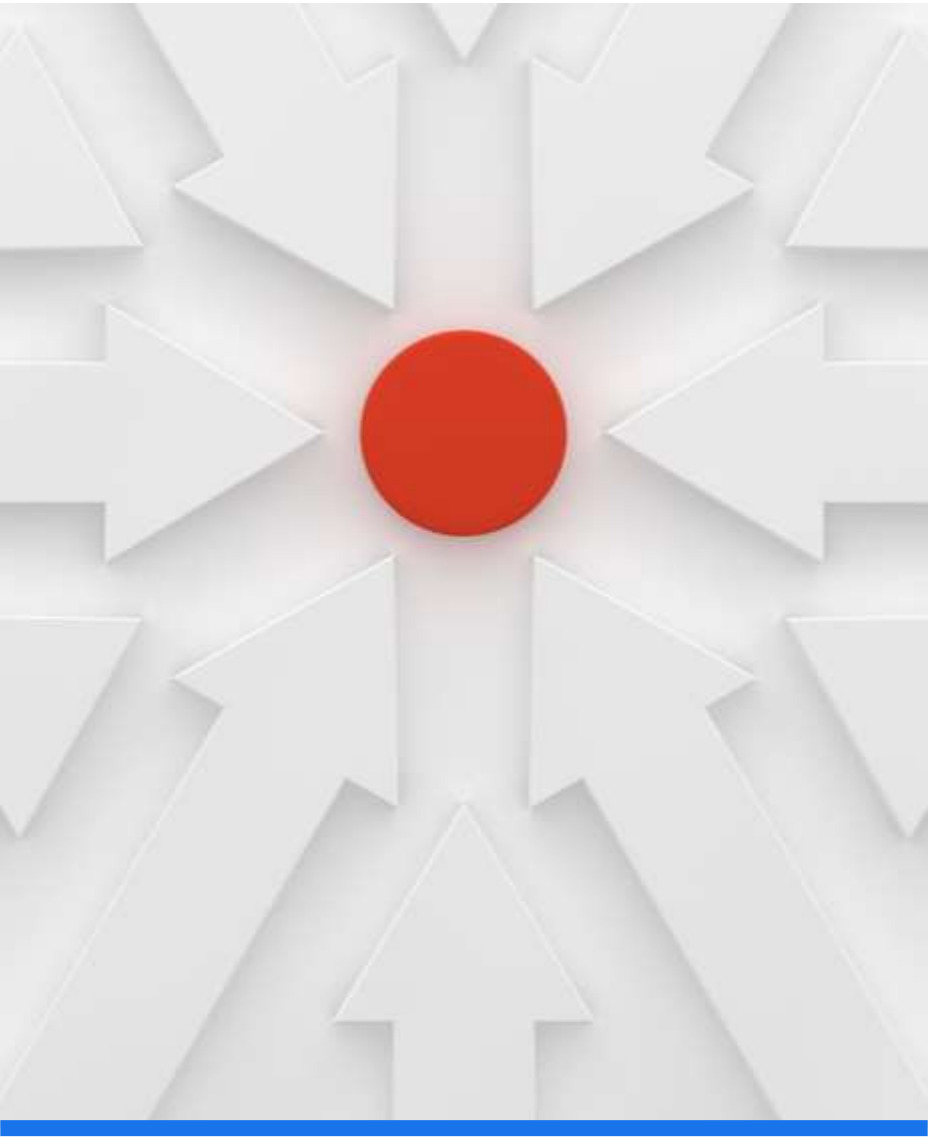
- **Proactive Improvement:** Organizations should actively seek opportunities to enhance processes, products, or services—not just respond to existing problems.
- **Sources of Opportunities:** These arise from technology advances, changing customer expectations, and evolving industry standards.

Examples:

- Cloud computing → lowers IT costs, increases scalability.
- Competitors' CRM use → motivates modernization.

Methods of Identification:

- **Internal analysis:** monitoring KPIs to find inefficiencies.
- **External benchmarking:** comparing with competitors or best practices.



2-Identifying Opportunities for Improvement

Types of Opportunities:

- **Incremental:** small, continuous improvements (e.g., automating manual tasks).
- **Transformational:** large-scale changes (e.g., AI-driven supply chain redesign).

- **Forward Thinking:** Proactive organizations ask how they can do better or what new technologies can enable—like early adopters of mobile banking gaining advantage.
- **Stakeholder Role:** Employees, customers, and management all contribute to identifying valuable opportunities through feedback and strategic evaluation.
- **Strategic Value:** Seeking opportunities turns system development into a strategic investment that fosters innovation, competitiveness, and long-term sustainability.

3-Submitting a System Request

- A system request is a structured document that proposes a new system or modification to an existing one
- **Once Need/Opportunity is** identified, a **system request** is formally submitted to propose a new system or modification.
- **It officially starts the System Development Life Cycle (SDLC) and turns informal ideas into formal proposals.**

Content of the Request:

1. Description of the problem or opportunity
2. Expected benefits
3. Possible constraints
4. Involved stakeholders

3-Submitting a System Request



Who Submits It: Usually department managers, end-users, or executive **sponsors**.



The request should highlight both **tangible benefits** (cost savings, efficiency, revenue growth) and **intangible benefits** (customer satisfaction, compliance).



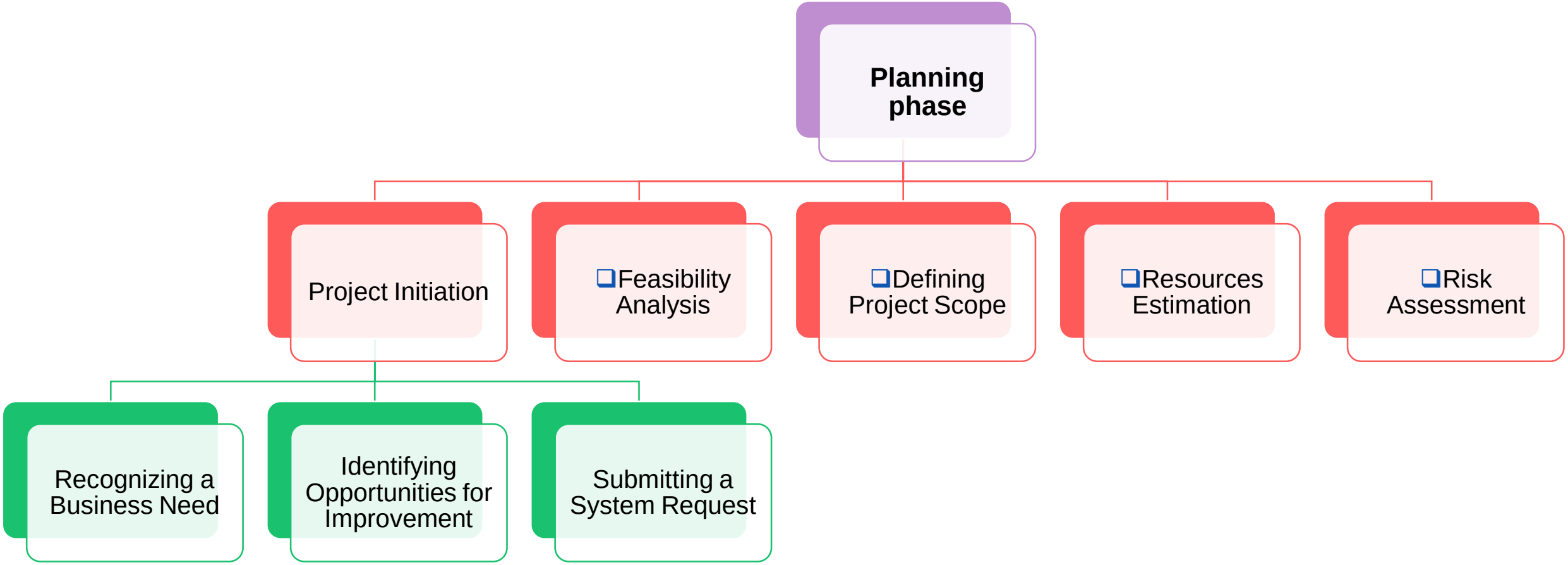
Example: A finance department might request an automated expense system to reduce manual errors.



Constraints: Budget, technology, or time limits should be documented early to assess project feasibility.



Identifying all affected parties (e.g., HR, IT, finance) ensures proper engagement and reduces resistance.


Planning phase
Project Initiation
❑Feasibility Analysis
❑Defining Project Scope
❑Resources Estimation
❑Risk Assessment
Recognizing a Business Need
Identifying Opportunities for Improvement
Submitting a System Request

B-Feasibility Study

1-The feasibility study is one of the most crucial steps in the planning phase of the SDLC.

2-Its purpose is to evaluate whether the proposed system is practical, cost-effective, and achievable within the organization’s resources and constraints.

Even if a system idea sounds appealing, it may fail due to technical limitations, excessive costs, legal concerns, or lack of organizational support.

B-Feasibility Study

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility
- Schedule Feasibility
- Legal and Ethical Feasibility

1-Technical Feasibility

Purpose: Technical feasibility evaluates whether the proposed system can be **developed and supported using the organization's existing technology, infrastructure, and expertise**.

IT Environment Analysis: Assess current hardware, software, and network capabilities to ensure they can support the new system.

Infrastructure Upgrades: Determine if upgrades are needed and whether they fit within budget limits.

Technical Expertise: Check if staff have the necessary skills; if not, consider hiring or outsourcing (e.g., for AI or machine learning projects).

System Compatibility: Ensure the new system integrates with or can replace existing legacy systems without causing disruptions.

Scalability: Verify that the system can handle future growth and increased demand (e.g., cloud solutions offer scalability advantages).

Confirms that the project is **technologically realistic and prevents investment in systems beyond t**he organization's technical capacity.

2-Economic Feasibility



Economic feasibility (or cost-benefit analysis) **determines if a system's financial benefits exceed its total costs.**



Purpose: Ensures the project provides solid financial justification before approval.



Cost Identification:



Development costs: hardware, software, labor



Operational costs: maintenance, training, support



Indirect costs: downtime during transition



Costs should be projected over the system's full lifespan.

2-Economic Feasibility

Benefit Estimation:

- **Tangible benefits:** reduced labor costs, faster transactions, increased sales
- **Intangible benefits:** better decisions, customer satisfaction, long-term value

- **Evaluation Methods:** Common techniques include **Net Present Value (NPV)**, **Return on Investment (ROI)**, and **Payback Period**.

Example: A $500,000 system that saves $1 million in two years is economically feasible.

- Analysts must check that cost and benefit estimates are realistic to avoid overruns or failure.

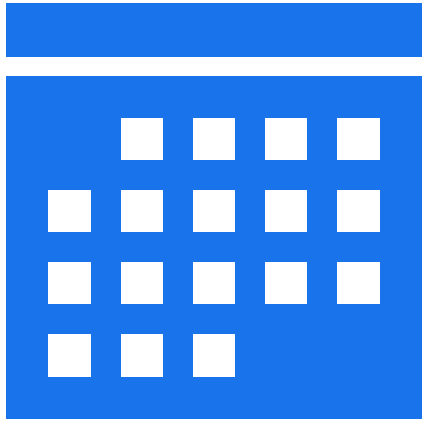
3-Operational Feasibility

Operational feasibility evaluates whether the proposed system will work effectively in the organization and be accepted by its users.

Even if a system is technically and financially sound, it can fail if employees resist using it.

- **Organizational Culture: Analysts assess how open** the organization is to change and whether it has a successful history of adopting new systems.
- **End-User Readiness: Evaluate employees' ability to use the system** and the amount of training required. Overly complex systems may slow adoption.
- **Management Support: Strong leadership backing is essential** to promote system adoption and allocate necessary resources.
- **Alignment with Goals:** The system must support the organization's strategic objectives (e.g., improving customer service).
- Ensures the **human and organizational factors** are considered so that people, processes, and technology work together for successful implementation.

4-Schedule Feasibility



Schedule feasibility assesses whether the project can be completed within the required time frame.

Importance: Even technically and financially sound projects can fail if not delivered on time.

Critical Deadlines: Identify key dates (e.g., launching before a major sales season) to align with business needs.

Time Estimation: Project managers estimate duration for all SDLC phases, including planning, testing, and training.

Realistic Scheduling: Avoid overly optimistic timelines that lead to stress and poor-quality results.

Resource Availability: Check if enough staff or resources are available; consider outsourcing or hiring if needed.

Risk and Contingency Planning: Account for potential delays, especially when using new or evolving technology.

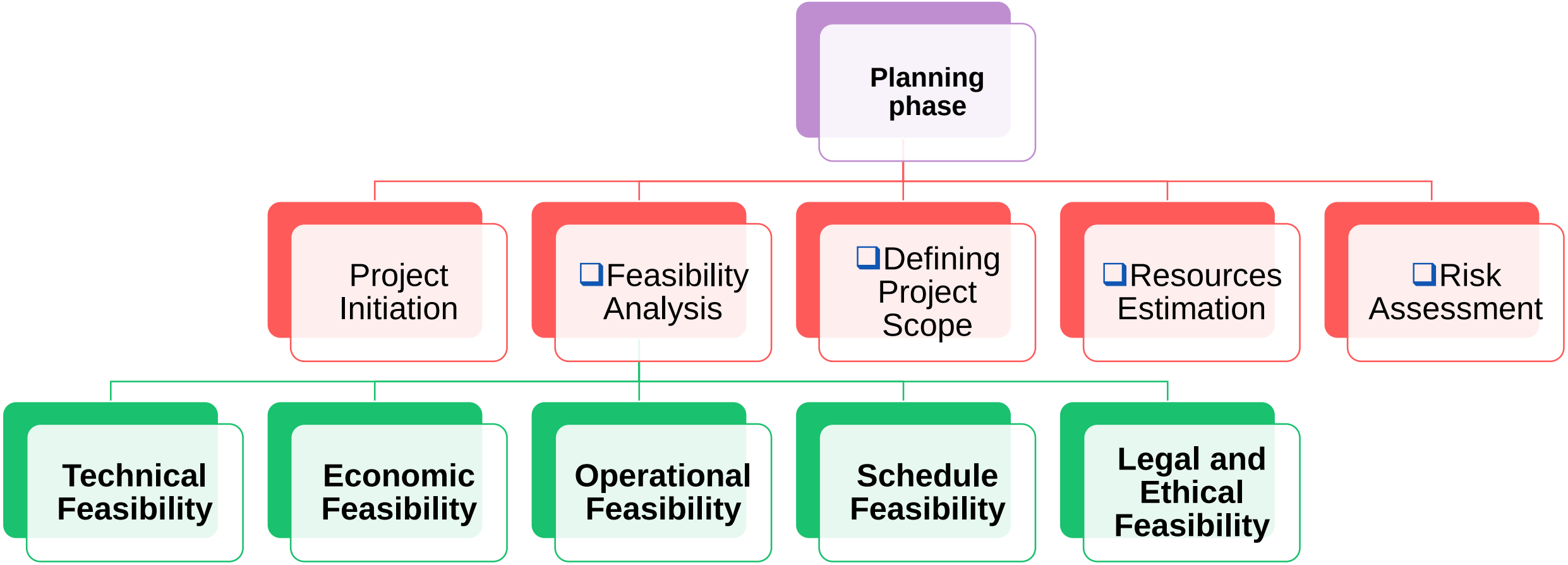
5-Legal and Ethical Feasibility

Legal and ethical feasibility ensures that the proposed system complies with laws, regulations, and ethical standards.

- Non-compliance can result in lawsuits, fines, or reputational damage.

Relevant Laws:

- **Healthcare:** must follow patient privacy laws (e.g., HIPAA).
- **Finance:** must comply with banking regulations.
- **Data protection:** must adhere to GDPR and similar laws.
- **Intellectual Property:** Ensure no infringement of patents, copyrights, or licenses; obtain proper agreements for third-party software.
- **Ethical Considerations:** Systems—especially AI-based ones—must avoid bias, discrimination, and unethical use of data.
- **Environmental Compliance:** Evaluate the system's energy usage and environmental impact, especially in regulated industries.


Planning phase
Project Initiation
❑Feasibility Analysis
❑Defining Project Scope
❑Resources Estimation
❑Risk Assessment
Technical Feasibility
Economic Feasibility
Operational Feasibility
Schedule Feasibility
Legal and Ethical Feasibility

C-Project Scope Definition

- The project scope acts as a **blueprint** that outlines the system's boundaries, objectives, deliverables, constraints, and assumptions.

- Without a well-defined scope, projects risk suffering from "scope creep"—a **common problem where uncontrolled changes or additions expand the project beyond its original goals, leading to delays, increased costs, and sometimes failure**.

- It prevents misunderstandings and provides measurable criteria for success.

C-Project Scope Definition

01

Objectives and Goals

02

Boundaries of the System

03

Deliverables and Outcomes

04

Constraints and Assumptions

1-Objectives and Goals

The first step in defining project scope is identifying the system's **objectives** (what it aims to achieve) and **goals** (how success will be measured).

Alignment with Strategy: Objectives should align with the organization's overall business strategy to ensure the system adds real value.

Example: If the objective is to improve customer satisfaction, the goal could be reducing inquiry response time by 50%.

SMART Goals: Goals should be **Specific, Measurable, Achievable, Relevant, and Time-bound** to ensure clarity and accountability.

Clear objectives and goals keep all stakeholders focused and guide decision-making throughout the SDLC.

Prioritization: Distinguish between **primary goals** (essential outcomes) and **secondary goals** (optional enhancements) to manage resources effectively.

2-Boundaries of the System

- Defining system boundaries specifies what the system **will** and **will not** include to prevent confusion and scope creep.
- **It e**nsures clear expectations, focused development, and effective resource management.

Key Questions to Set Boundaries:

- What processes will the system automate?
- Which departments or users will interact with it?
- What existing systems will it integrate with?
- What functionalities are out of scope?

Example: A payroll system may include salary processing and tax deductions but exclude recruitment and training.

- **Geographic Scope:** Define where the system will be implemented (e.g., one branch initially, later expansion).
- **Documenting exclusions** (e.g., “will not handle inventory”) **prevents uncontrolled feature requests and prevent scope creeping**
- **Resource Planning:** Clear boundaries help allocate time, budget, and staff efficiently.



3-Deliverables and Outcomes

- **Deliverables:** **Tangible products or results** produced by the project.
- The benefits or organizational improvements resulting from those deliverables.
- Documenting both ensures accountability and provides clear success measures.
- Each deliverable must have measurable standards (e.g., “system must process 10,000 transactions per hour without errors”).

Types of Deliverables:

- **Short-term:** Documents, prototypes, test results.
- **Long-term:** Final deployed system.

3-Deliverables and Outcomes

Examples of Outcomes:

- Improved customer interaction tracking → higher customer retention (as in a CRM system).

Defining deliverables and outcomes gives a clear roadmap of expected results and performance benchmarks, keeping stakeholders engaged and aligned throughout the project.

4- Constraints and Assumptions

Constraints: Limitations the project must operate within.

Assumptions: Conditions believed to be true during planning but not guaranteed.

Documenting both ensures realistic expectations, clarifies project limits, and supports proactive risk management.

Examples of Constraints:

Budget: Total cost capped at $1 million.

Time: System must launch within six months.

Resources: Limited skilled developers.

4- Constraints and Assumptions

Technology: Must use existing database infrastructure.

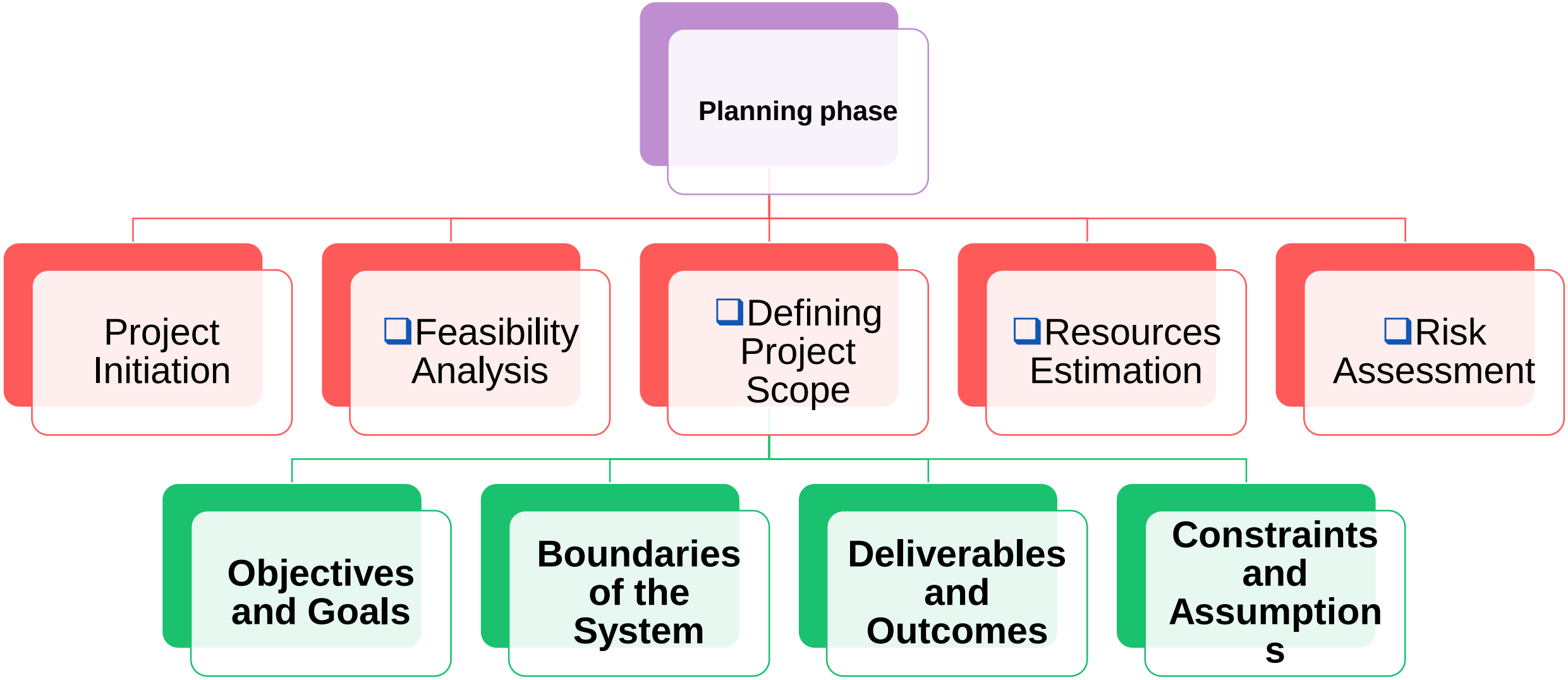
Examples of Assumptions:

End-users will be available for training.

Existing hardware can support the new system.

Regulations will not change during development.

Risk Consideration: If assumptions later prove false, the project may face delays or increased costs.


Planning phase
Project Initiation
❑Feasibility Analysis
❑Defining Project Scope
❑Resources Estimation
❑Risk Assessment
Objectives and Goals
Boundaries of the System
Deliverables and Outcomes
Constraints and Assumptions

Resource Estimation

Resource estimation is one of the most crucial components of the planning stage in system development.

It involves determining the **types, quantities, and timing** **of** **resources** needed to successfully execute the project.

Types of Resources:

- **Human resources:** staff and required skills
- **Financial resources:** budget and costs
- **Technological resources:** hardware and software
- **Time resources:** schedules and deadlines

Resource Estimation

Ensures the project is achievable within defined constraints and maintains quality standards.

Overestimation: leads to unnecessary costs and inefficiency.

Underestimation: causes delays, employee burnout, and potential project failure.

Accurate resource estimation supports effective planning, cost control, and timely project delivery.

Human Resource Estimation



Human resource estimation determines the **number of people, their roles, and required skills** for a project.

Process Steps:

Analyze the **Work Breakdown Structure (WBS)** to identify all project tasks.

Estimate **staff quantity**, **expertise level**, and **time** for each task.

Roles Typically Required: Project manager, business analysts, system designers, developers, database administrators, testers, and trainers.

Key Considerations:

Availability of skills: If internal staff lack expertise, external consultants may be needed.

Productivity variations: Use historical data or benchmarks to improve accuracy.

Training and knowledge transfer: Include time and resources for end-user and IT staff training.

It Ensures the right team composition, prevents role overlap, manages costs, and supports smooth system implementation.

Financial Resource Estimation

Financial estimation determines the total budget needed for a project, covering both **direct costs** (e.g., salaries, hardware, software) and **indirect costs** (e.g., administration, training, maintenance).

It usually begins with a **cost breakdown** of each task in the WBS to assign specific cost estimates.

Common Techniques: Analogous Estimation: Based on historical project data.

Parametric Estimation: Uses formulas like cost per line of code or function point.

Bottom-Up Estimation: Aggregates individual task costs to get the total budget.

Contingency Reserves: Allocate 10–20% extra for unexpected risks (e.g., vendor delays or scope changes).

Lifecycle Costing: Includes long-term costs for maintenance, upgrades, and support.

Technological Resource Estimation

Technological resource estimation identifies the **hardware, software, and infrastructure** required to support the system.

Hardware Estimation: Includes servers, storage, workstations, and network equipment sized according to user load and transaction volume (e.g., large-scale e-commerce vs. small HR systems).

Software Estimation: Covers programming tools, DBMS, testing platforms, and collaboration software—considering **licensing costs**, **subscription models**, and **system compatibility**.

Infrastructure Services: Estimates resources for **cloud hosting**, such as storage, processing power, and bandwidth. Since cloud costs are usage-based, accurate projections help avoid overspending.

Security Requirements: Must include costs for **firewalls**, **encryption**, and **monitoring tools** to maintain cybersecurity compliance.

System Integration: Estimation should address **compatibility** with existing systems, including the need for **middleware** or **upgrades** to ensure smooth integration.

Time Resource Estimation

Time estimation determines the **total duration of a project** and ensures alignment with stakeholder expectations.

HOW: Begins with the **Work Breakdown Structure (WBS)** and identification of **task dependencies**—some tasks run in parallel, while others are sequential (e.g., testing follows coding).

Tools: Techniques like **Gantt charts** and the **Critical Path Method (CPM)** help visualize dependencies and schedule flow.

Effort Calculation: Tasks are estimated in **person-hours/days**; team size and availability influence the actual duration.

Risk and Buffering: Account for **uncertainties** (e.g., delays in hardware delivery or changing requirements) by adding **time buffers**.

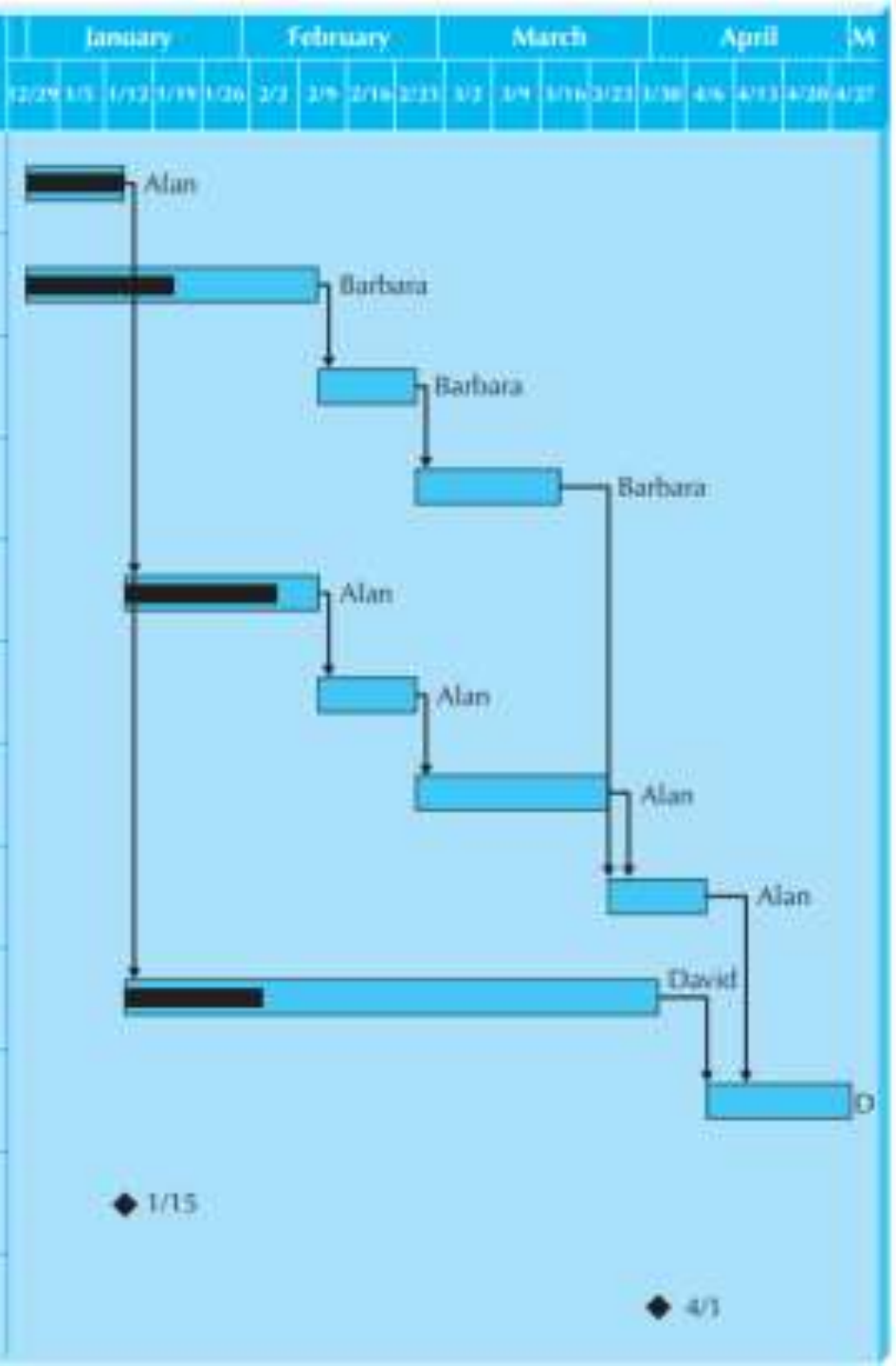
Business Alignment: The schedule must support **organizational deadlines** such as product launches or regulatory requirements, where time is a non-flexible constraint.

Work Breakdown Structure

Identifying Tasks

Task Number	Task Name	Duration (in weeks)	Dependency	Status
1	Identify vendors	2		Complete
2	Review training materials	6	1	Complete
3	Compare vendors	2	2	In Progress
4	Negotiate with vendors	3	3	Open
5	Develop communications information	4	1	In Progress
6	Disseminate information	2	5	Open
7	Create and administer survey	4	6	Open
7.1	Create initial survey	1		Open
7.2	Review initial survey	1	7.1	Open
7.2.1	Review by Director of IT Training	1		Open
7.2.2	Review by Project Sponsor	1		Open
7.2.3	Review by Representative Trainee	1		Open
7.3	Pilot test initial survey	1	7.1	Open
7.4	Incorporate survey changes	1	7.2, 7.3	Open
7.5	Create distribution list	0.5		Open
7.6	Send survey to distribution list	0.5	7.4, 7.5	Open
7.7	Send follow-up message	0.5	7.6	Open
7.8	Collect completed surveys	1	7.6	Open
8	Analyze results and choose vendor	2	4, 7	Open
9	Build new classrooms	11	1	In Progress
10	Develop course options	3	8, 9	Open

ID	Task Name	Duration	Start	Finish	Prede
1	Identify vendors	2 wks	Wed 1/1/10	Tue 1/14/10	
2	Review training materials	6 wks	Wed 1/1/10	Tue 2/11/10	
3	Compare vendors	2 wks	Wed 2/12/10	Tue 2/25/10	2
4	Negotiate with vendors	3 wks	Wed 2/26/10	Tue 3/18/10	1
5	Develop communications information	4 wks	Wed 1/15/10	Tue 2/11/10	1
6	Disseminate information	2 wks	Wed 2/12/10	Tue 2/25/10	5
7	Create and administer survey	4 wks	Wed 2/26/10	Tue 3/25/10	6
8	Analyze results and choose	2 wks	Wed 3/26/10	Tue 4/8/10	4, 7
9	Build new classroom	11 wks	Wed 1/15/10	Tue 4/1/10	1
10	Develop course options	3 wks	Wed 4/9/10	Tue 4/29/10	8, 9
11	Budget Meeting	1 day	Wed 1/15/10	Wed 1/15/10	
12	Software Installation	1 day	Tue 4/1/10	Tue 4/1/10	



Gantt chart

Tools and Techniques for Estimation



Purpose: Tools and techniques enhance the **accuracy** of resource estimation in system development projects.



Function Point Analysis (FPA): Estimates resources based on system functionality delivered to users.



COCOMO (Constructive Cost Model): Uses mathematical models to predict effort based on software size and complexity.



Software Tools: **Microsoft Project**, **Primavera**, and **JIRA** help plan tasks, assign resources, and forecast costs and schedules automatically.



Expert Judgment: Experienced professionals contribute realistic estimates based on prior projects.



Hybrid Approach: Combining **quantitative models** with **expert input** minimizes bias and yields more accurate, balanced estimates.

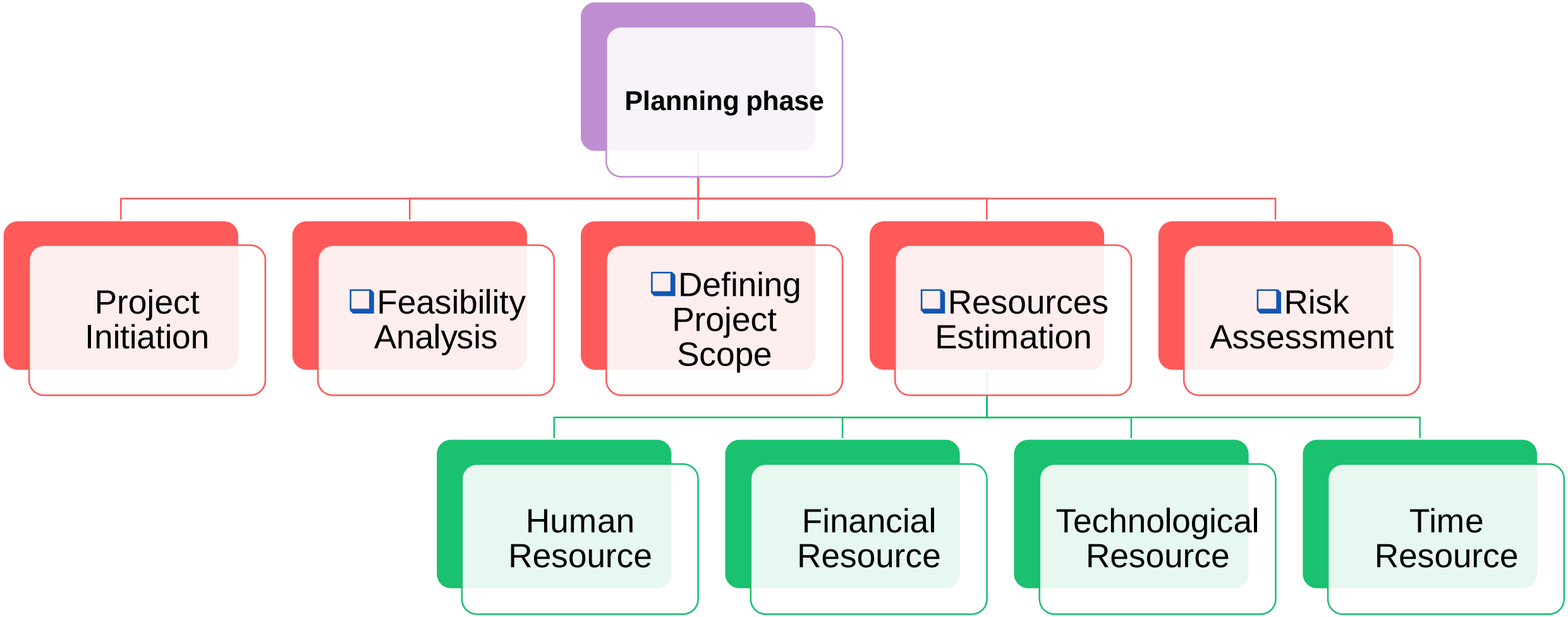
Challenges in Resource Estimation

Estimation Challenges:

- **Incomplete requirements:** Unclear scope leads to inaccurate or underestimated time and cost.
- **Scope creep:** Adding new features after estimation causes resource shortages.
- **Technological uncertainties:** New tools or systems often bring hidden learning curves and integration issues.
- **Human factors:** Differences in productivity, turnover, and conflicts reduce predictability.
- **External dependencies:** Vendor delays or regulatory changes can disrupt schedules.

Mitigation Strategies:

- Regularly **review and update estimates** as the project evolves.
- Maintain **flexibility and adaptability** to respond to new challenges and information.


Planning phase
Project Initiation
❑Feasibility Analysis
❑Defining Project Scope
❑Resources Estimation
❑Risk Assessment
Human Resource
Financial Resource
Technological Resource
Time Resource

See You Again

